

Celfras Standard Protocol

V3.1.0

Celfras Standard Protocol — Part I: 표준

이 부(Part)는 모든 Celfras 제품이 구현하는 유선 프로토콜을 정의한다. 즉 링크 위에서 바이트를 어떻게 프레임화하는지, 요청에 어떻게 응답하는지, 핵심 명령이 무엇인지, 그리고 어떤 구현이 적합(conforming)하기 위해 무엇을 해야 하는지를 정의한다. 이 부는 그 자체로 완결적이다. 어떤 제품의 소스에도 접근하지 않고 이 부만 가지고 디바이스나 호스트를 작성할 수 있다.

Part II 는 요구사항이 아니라 기록이다. 이 표준이 제품에 맡겨 둔 부분 — 식별자 맵, 선택적 명령 그룹, 계층별 오류 코드 — 을 개별 제품이 어떻게 채웠는지 문서화한다. Part II 의 어떤 내용도 Part I 의 요구를 완화하지 않는다.

1. 범위

1.1 이 부가 정의하는 것

- 적합한 링크가 동작하는 전송 수단(§ 2).
- 프레임 형식, 그 COBS 인코딩, 무결성 검사, 그리고 수신 측의 검증 규칙(§ 3).
- 요청/응답 모델, 상태 바이트, 핵심 오류 공간(§ 4).
- 동작 모드, 그리고 각 모드에서 어떤 명령에 응답하는지(§ 5).
- 핵심 명령 집합과 표준화된 명령 그룹(§ 6, § 7, § 8).
- 오피코드(opcode) 공간, 예약된 영역, 제품이 이를 확장하는 방법(§ 9).
- 적합성 점검표(§ 10).

1.2 이 부가 의도적으로 정의하지 않는 것

제품마다 정당하게 달라지는 모든 것: 변수 명령과 파라미터 명령 뒤에 있는 식별자 맵(§ 8), 어떤 텔레메트리 필드 비트가 실제 측정값을 실는지(§ 7.2), 튜닝 식별자의 의미(§ 6.4), 그리고 제품이 내부에 굳혀 넣는 모든 임계값. 또한 명시적으로, 전송 수단의 구현도 정의하지 않는다 — § 2 를 보라.

1.3 역할

호스트(Host) — PC 측 도구. 요청을 발행하고 응답과 비요청 프레임을 소비한다.

디바이스(Device) — 제품 펌웨어. 요청에 응답하며 비요청 프레임을 보내어도 된다.

브리지(Bridge) — 같은 링크 위에 끼어 들어가는 선택적 중계기. 자신에게 예약된 오피코드(§ 9.2)에는 스스로 응답하고, 그 밖의 모든 것은 손대지 않고 디바이스로 전달한다. 브리지는 적합한 호스트와 디바이스에 대해 투명하다. 브리지에만 다르게 적용되는 규칙은 정확히 하나뿐이다. 브리지는 검증에 실패한 프레임을 전달하여서는 안 되며, 그러한 프레임을 조용히 폐기하는 대신 응답하여도 된다(§ 3.7).

하나의 링크에는 정확히 하나의 호스트와 하나의 디바이스가 참여한다. 브리지가 있더라도 어느 방향의 프레임 형식도 바뀌지 않는다.

1.4 요구 표현

이 표준은 요구의 강도를 다음 표현으로만 나타내며, 문서 전체에서 이 대응을 그대로 유지한다.

~**하여야 한다** (MUST) — 이 표준의 절대적 요구사항이다. 이를 위반한 구현은 적합하지 않다. ~**하여서는 안 된다** (MUST NOT) — 이 표준의 절대적 금지이다. 이를 위반한 구현은 적합하지 않다. ~**하는 것을 권장한다** / ~**하지 않을 것을 권장한다** (SHOULD / SHOULD NOT) — 사유를 밝힌 경우에 한하여 벗어날 수 있는 요구사항이다. 벗어난 제품은 그 사실을 Part II 에 기록한다. ~**하여도 된다** (MAY) — 실제로 선택 사항이다.

규범적 허용은 "~하여도 된다" 로만 표현한다. "~할 수 있다" 는 허용이 아니라 능력이나 가능성을 서술할 때에만 쓴다.

1.5 바이트 순서와 표기

표에서 달리 정하지 않는 한 모든 다중 바이트 필드는 리틀 엔디언(little-endian) 이며 부호가 없다. 비트 0 은 최하위 비트이다. u8, u16, u32 는 각각 그 폭의 부호 없는 정수를 뜻한다. 바이트 값은 16진수로 적는다.

2. 전송

링크는 비동기 시리얼이다. 데이터 8비트, 패리티 없음, 정지 비트 1, 흐름 제어 없음. 기본 속도는 1 Mbaud 이다. 제품은 다른 속도를 문서화하여도 되지만, 양쪽 끝은 그 속도를 대역 외에서 합의하여야 한다 — 프로토콜 안에는 속도를 협상하는 수단이 없다.

링크는 전이중이며, 양방향 모두 같은 프레임 형식(§ 3)을 쓴다. 디바이스는 언제든지 비요청 프레임을 보내어도 된다(§ 7). 자신의 요청과 그에 대한 응답 사이에 비요청 프레임이 끼어드는 것을 견디지 못하는 호스트는 적합하지 않다.

전송 백엔드는 이 표준의 일부가 아니다. 디바이스가 자신의 UART 를 DMA 로 처리하는지, 바이트 단위 인터럽트로 처리하는지, 폴링으로 처리하는지는 구현 선택이다. 선 위에서는 이들을 구별할 수 있는 것이 아무것도 없으며, 호스트는 어느 쪽이 쓰이는지에 의존하여서는 안 된다. 참조 구현은 DMA 백엔드를 출하하고 빌드 시점에 선택 가능한 인터럽트 백엔드를 함께 가지고 있다. 둘은 링크 위에서 바이트 단위로 동일하며, 둘 다 적합하다. 링 버퍼, 송신 큐, 인터럽트 우선순위, DMA 채널에 관한 서술은 제품 자신의 문서에 속하며, 이 표준에 속하지 않는다.

§ 3.4 의 프레임িং은 자기 동기화되므로, 수신 측은 잘리거나 손상된 프레임으로부터 대역 외 리셋 없이 복구하여야 하며, 프레임 간 간격 없이 연달아 도착하는 프레임을 받아들여야 한다. 디바이스는 프레임 앞에 유틸리티나 라인 브레이크, 프리앰블을 요구하여서는 안 된다.

3. 프레임 형식

3.1 디코딩된 프레임

모든 프레임은 방향에 관계없이 § 3.4 의 프레임িং을 벗겨 내면 다음 구조를 가진다.

```
[LEN u16 LE][CMD u8][SEQ u8][PAYLOAD ...][CRC16 u16 LE]
```

필드	폭	의미
LEN	u16 LE	자신의 뒤에 오는 바이트 수: 1 (CMD) + 1 (SEQ) + len(PAYLOAD) + 2 (CRC16).

필드	폭	의미
CMD	u8	오피코드. 응답은 자신이 답하는 요청과 같은 오피코드를 싣는다.
SEQ	u8	대응 토큰. § 3.5 를 보라.
PAYLOAD	LEN - 4	명령마다 다르며 비어 있을 수 있다. 응답에서는 항상 [STATUS][DATA...] 이다. § 4 를 보라.
CRC16	u16 LE	무결성 검사. § 3.3 을 보라.

디코딩된 프레임의 전체 길이는 $2 + \text{LEN}$ 이다. 따라서 가장 짧은 적법한 프레임은 여섯 바이트이다. $\text{LEN}(2) + \text{CMD} + \text{SEQ} + \text{CRC16}(2)$ 에 빈 페이로드.

3.2 LEN 이 세는 것

LEN 은 LEN 필드 **자신의 뒤에 오는** 모든 것을 세며, 여기에는 CRC 두 바이트가 포함된다. 자기 자신은 세지 **않는** 다. 수신 측은 LEN 을 실제로 디코딩한 길이와 대조하여야 하며, 둘이 어긋나는 프레임은 폐기하여야 한다 — 이 검사는 하한 검사가 아니라 일치 검사이다.

3.3 무결성 검사

CRC16 은 **CRC-16/CCITT-FALSE** 이다.

항목	값
폭	16 비트
다항식	$0x1021$
초깃값	$0xFFFF$
입력 반전	없음
출력 반전	없음
최종 XOR	없음

검사 범위: 디코딩된 프레임의 LEN 첫 바이트부터 PAYLOAD 마지막 바이트까지. CRC 두 바이트 자신은 제외된다. CRC 는 디코딩된 프레임 위에서 계산하며, § 3.4 의 인코딩된 바이트 위에서 계산하지 않는다.

이것이 상호 운용에서 가장 흔한 실수이며, 서로 독립적인 두 가지가 모두 맞아야 한다. CRC 변종(variant), 그리고 그 검사 범위(coverage). 먼저 변종을 다음 벡터로 확인하라.

입력	CRC16
31 32 33 34 35 36 37 38 39 (ASCII "123456789")	$0x29B1$
01 00	$0x2E3E$
(빈 입력)	$0xFFFF$

"123456789" 에 대한 $0x29B1$ 은 이 변종의 공표된 검사값이다. 다른 값을 내놓는 구현은 변종이 틀린 것이며 — 대개는 반전(reflection)과 최종 XOR 이 원인이다 — 검사 범위를 아무리 조정해도 고쳐지지 않는다. 변종이 확인되면 § 3.6 의 실제 프레임 예제로 검사 범위를 확인하라.

3.4 프레임

디코딩된 프레임은 COBS(Consistent Overhead Byte Stuffing)로 인코딩되며, 프레임 구분자로 0x00 한 바이트를 뒤에 붙인다.

```
wire bytes = COBS_encode(decoded frame) + 0x00
```

COBS 는 인코딩된 바이트에 0x00 이 들어가지 않음을 보장하므로 구분자는 모호하지 않다. 구분자는 인코딩된 데이터의 일부가 **아니며** CRC 의 검사 범위에도 **들어가지 않는다**.

수신 측은 0x00 을 무조건적인 프레임 경계로 취급하여야 한다. 즉 0x00 이 도착할 때까지 바이트를 모으고, 모인 것을 디코딩한 다음, 새 프레임을 시작한다. 링크가 자기 동기화되는 것은 바로 이 때문이다 — 어떤 손상이 있더라도 다음 0x00 이 프레임 정렬을 되돌린다. 수신 측은 길이가 0 이거나 그 밖에 디코딩할 수 없는 누적분을 프레임으로 취급하지 말고 폐기하여야 한다.

이 인코딩은 Cheshire 와 Baker 가 발표한 COBS 이며, 그 전부를 아래에 서술한다. 적합한 구현에는 이 밖의 것이 필요하지 않으며, 특히 "0 이 아닌 바이트를 세어 그 개수를 내보낸다" 는 한 줄 요약만 보고 작성하여서는 안 된다. 그렇게 하면 짧은 프레임에는 맞고 긴 프레임에는 틀린 인코더가 나오며, § 3.4.1 의 0xFF 규칙이 그 차이이다.

3.4.1 인코딩

인코더는 디코딩된 프레임을 훑으며 **그룹** 으로 나눈다. 각 그룹은 다음 중 하나이다.

- 0 이 아닌 바이트 n 개의 연속($0 \leq n \leq 253$)이며, 디코딩된 프레임의 0x00 바이트로 끝나는 그룹.
- 0 이 아닌 바이트 정확히 254 개의 연속이며, 아무것으로도 끝나지 않는 그룹.
- 0 이 아닌 바이트 n 개의 마지막 연속($0 \leq n \leq 253$)이며, 디코딩된 프레임의 끝으로 끝나는 그룹.

각 그룹은 **코드 바이트** 하나와 그 그룹의 0 이 아닌 바이트들을 순서대로 그대로 이어 내보낸다. 코드 바이트는 $n + 1$ 이므로 0x01-0xFF 범위에 있으며 절대 0x00 이 되지 않는다. 그룹을 끝낸 디코딩된 프레임의 0x00 은 내보내지 **않는다**. 코드 바이트가 그 자리를 대신하며, 이것이 0 을 선 위에서 없애는 방식이다.

따라서 0xFF 코드는 다른 모든 값과 의미가 다르다.

- 0x01-0xFE — code - 1 개의 0 이 아닌 바이트가 뒤따르며, 디코딩된 프레임에서 **그 뒤에 0x00 이 있었다**(마지막 그룹이라면 거기서 프레임이 끝났다).
- 0xFF — 254 개의 0 이 아닌 바이트가 뒤따르며, **그 뒤에 0x00 은 없었다**. 코드 바이트가 그보다 큰 수를 셀 수 없기 때문에 그룹이 끝났을 뿐이다.

인코더는 0 이 아닌 바이트 254 개가 완전히 채워질 때마다 0xFF 코드를 내보내야 하며, 그 다음 새 그룹으로 이어 가야 한다. 0 이 없는 긴 연속을 나눌 때 프레임에 없던 0 을 만들어 넣어서는 안 된다. 또한 인코더는 마지막 그룹에 데이터 바이트가 하나도 없더라도 그 그룹의 코드 바이트를 내보내야 한다. 254 바이트 경계에서 정확히 끝나는 프레임은 0xFF 그룹 뒤에 0x01 하나만 놓인 형태로 인코딩된다.

인코딩된 프레임은 디코딩된 프레임보다 한 바이트 길며, 0xFF 코드 하나마다 한 바이트씩 더 길어진다 — n 바이트 프레임에서 0xFF 코드는 많아야 $\text{floor}(n / 254)$ 개이다. 여기에 § 3.4 의 구분자가 한 바이트를 더한다.

3.4.2 디코딩

디코더는 누적된 바이트 — 직전 구분자 이후 수신한 모든 것, 구분자 자신은 제외 — 를 다음과 같이 처리하여야 한다.

1. 코드 바이트 하나를 읽는다. 값이 $0x00$ 인 코드 바이트는 거부하여야 한다. 인코더는 그런 값을 만들지 않으므로 그 누적분은 프레임이 아니다.
2. 이어지는 code - 1 바이트를 그대로 출력으로 복사한다. 남은 바이트가 code - 1 개보다 적으면 프레임을 거부하여야 한다.
3. 코드가 $0xFF$ 가 **아니었고 그리고** 아직 바이트가 남아 있다면, 출력에 $0x00$ 한 바이트를 덧붙인다. 코드가 $0xFF$ 였거나 남은 바이트가 없다면 아무것도 덧붙이지 않는다.
4. 바이트가 남아 있으면 1 단계로 돌아간다.

디코더는 부분적으로 디코딩된 프레임을 위로 올려보내지 말고 프레임 전체를 거부하여야 한다.

3 단계는 두 코드 의미의 정확한 역연산이며, 두 조건 모두 무게를 가진다. $0xFF$ 그룹 뒤에 0 을 덧붙이면 송신 측이 보내지 않은 바이트를 끼워 넣는 것이 된다. *마지막* 그룹 뒤에 덧붙이면 프레임의 끝에 지나지 않았던 것을 바이트로 만드는 것이 된다. 어느 쪽이든 디코딩된 길이가 한 바이트 이상 어긋나므로 프레임은 § 3.7 의 LEN 검사와 CRC 검사에 걸려 폐기된다 — 즉 잘못된 디코더는 디코딩 실패가 아니라 **긴 프레임에서의 CRC 실패** 로 드러난다. 그 증상을 쫓는 구현자는 CRC 를 다시 확인하기 전에 3 단계를 먼저 확인할 것을 권장한다.

3.5 SEQ

SEQ 는 응답을 그 요청과 대응시킨다.

- 요청에 대한 응답은 그 요청의 SEQ 를 그대로 되돌려주어야 한다.
- 어떤 요청에도 답하지 않는 디바이스 → 호스트 방향의 비요청 프레임은 $SEQ = 0$ 을 사용하여야 한다.
- SEQ 는 디바이스에게 불투명하다. 디바이스는 SEQ 를 해석하거나 검증하여서는 안 되며, 요청마다 값이 바뀔 것을 요구하여서도 안 된다. SEQ 값을 할당하고 중복이나 누락을 검출하는 일은 전적으로 호스트의 몫이다.

요청을 파이프라이닝하는 호스트는 SEQ 로 응답을 맞추어야 하며, 응답이 요청을 보낸 순서대로 도착한다고 가정하여서는 안 된다.

비요청 프레임이 자신을 식별하는 방법이 $SEQ = 0$ 이므로, 스트리밍(§ 7)을 사용하는 링크에서 호스트는 $SEQ = 0$ 인 요청을 발행하지 않을 것을 권장한다. 그렇지 않으면 같은 명령에 대해 호스트가 시작한 정지와 디바이스가 시작한 정지를 구별할 수 없다 (§ 7.2).

3.6 실제 프레임 예제

이 절의 모든 바이트는 참조 구현에 대해 실제로 계산한 값이며, 예시용으로 지어낸 값이 아니다.

3.6.1 여섯 바이트 요청

$SEQ = 0x07$ 이고 페이로드가 없는 완전한 CMD_PING 요청.

디코딩된 프레임 — 여섯 바이트:

```
04 00 01 07 E7 0D
```

바이트	값	필드
0-1	04 00	LEN = 4 — CMD + SEQ + 페이로드 0 바이트 + CRC16
2	01	CMD — § 6.1 의 오피코드
3	07	SEQ — 호스트의 대응 토큰. 응답이 이 값을 되돌려준다
—	—	PAYLOAD — 이 명령에서는 비어 있다
4-5	E7 0D	CRC16 = 0x0DE7, 리틀 엔디언

CRC 는 바이트 0-3, 즉 04 00 01 07 위에서 계산한다 — LEN 두 바이트를 **포함하고** CRC 두 바이트를 **제외한** 다. CMD 부터 시작하는 구현은 다른 값을 얻으며, 그 프레임은 오류 응답 없이 폐기된다(§ 3.7).

COBS 인코딩과 구분자를 거친 선 위의 모습 — 여덟 바이트:

02 04 05 01 07 E7 0D 00

바이트	값	의미
0	02	COBS 그룹 코드: 0 이 아닌 바이트 한 개가 뒤따르고, 그 자리에 0 이 있었다
1	04	LEN 하위 바이트
2	05	COBS 그룹 코드: 0 이 아닌 바이트 네 개가 뒤따른다
3-6	01 07 E7 0D	CMD, SEQ, CRC16 — 그대로 실려 간다
7	00	프레임 구분자 — COBS 데이터가 아니고 CRC 범위에도 들어가지 않는다

COBS 가 한 일을 보라. 디코딩된 프레임 안의 유일한 0x00(LEN 상위 바이트)이 없어졌고, 그 자리를 대신한 코드 바이트가 0x02 이다 — 데이터 한 바이트, 그 다음 0(§ 3.4.1). 마지막 코드는 0x05 이다. 데이터 네 바이트, 그 다음 프레임의 끝.

3.6.2 254 바이트 연속이 있는 프레임

위 프레임으로는 0xFF 코드를 보일 수 없다. 여섯 바이트 안에는 긴 연속이 없기 때문이다. 이 예제는 보일 수 있다. 이것은 펌웨어 이미지 데이터 요청이며, 0 이 없는 긴 연속이 실제로 나타나는 곳이다. 이미지 바이트는 임의의 값이라 0 없이 수백 바이트가 이어지는 일이 흔하다. 이 명령의 오피코드는 브리지 계층에 예약된 대역 (§ 9.2)에 속하며 Part II 에 정의되어 있다 — 이 절의 프레임িং은 두 대역에서 동일하고, 이 예제가 바로 그것을 실행해 보이는 경우이다.

디코딩된 프레임은 266 바이트이다.

바이트	값	필드
0-1	08 01	LEN = 264
2	E9	CMD
3	09	SEQ
4-7	00 10 00 00	이미지 오프셋, u32 LE
8-263	01 02 03 ... FD FE 01 02	이미지 256 바이트. 바이트 8 + i 는 (i mod 254) + 1 을 담으므로 이 연속에는 0 인 바이트가 없다

바이트	값	필드
264-265	1A 86	CRC16 = 0x861A, 리틀 엔디언

인코딩하면 268 바이트, 구분자를 포함하면 269 바이트이며, COBS 그룹 다섯 개로 나뉜다.

인코딩된 바이트	코드	데이터 바이트	코드가 말하는 것
0-4	05	08 01 E9 09	0 이 아닌 네 바이트, 그 다음 0x00 — 디코딩된 바이트 4
5-6	02	10	0 이 아닌 한 바이트, 그 다음 0x00 — 디코딩된 바이트 6
7	01	—	데이터 바이트 없음, 그 다음 0x00 — 디코딩된 바이트 7
8-262	FF	01 02 ... FD FE (254 바이트)	0 이 아닌 254 바이트, 그리고 그 뒤에 0 은 없다
263-267	05	01 02 1A 86	0 이 아닌 네 바이트, 그 다음 프레임의 끝
268	—	00	프레임 구분자 — COBS 데이터가 아니고 CRC 범위에도 들어가지 않는다

따라서 인코딩된 첫 바이트들은 05 08 01 E9 09 02 10 01 FF 01 02 03 04 ... 이고, 마지막 바이트들은 ... FB FC FD FE 05 01 02 1A 86 00 이다.

인코딩된 바이트 8 의 0xFF 가 이 예제의 요점이다. 그 254 개의 데이터 바이트는 디코딩된 바이트 8-261 이며, 뒤따르는 그룹은 그 사이에 아무것도 없이 디코딩된 바이트 262 에서 이어진다. 거기에 0x00 을 끼워 넣는 디코더는 266 바이트가 아니라 267 바이트를 만들어 내며, 온전하게 도착한 그 프레임은 LEN 검사와 CRC 검사에 모두 걸려 아무 응답 없이 폐기된다.

마지막 그룹도 보라. 그 코드는 0xFF 가 아니라 0x05 이고, 싼고 있는 네 바이트 뒤에는 0 이 아니라 프레임의 끝 이 온다. 디코딩된 바이트 265 는 CRC 의 상위 바이트이며 그 뒤에는 아무것도 없다. 마지막 그룹 뒤에 0 을 덧붙이는 디코더는 같은 방식으로, 같은 한 바이트만큼 틀린다.

3.7 수신 측 검증, 순서대로

수신 측은 다음 검사를 이 순서대로 적용하여야 하며, 처음 실패하는 지점에서 프레임을 폐기하여야 한다.

1. COBS 디코딩(§ 3.4.2)이 성공하고 최소 여섯 바이트를 내놓는다.
2. 디코딩된 프레임에서 마지막 두 바이트를 제외한 부분에 대한 CRC 가 그 두 바이트가 싼고 있는 CRC16 과 같다.
3. LEN 이 디코딩된 길이에서 2 를 뺀 값과 같다.

이 중 하나라도 실패한 프레임은 폐기하여야 하며, 전달하여서는 안 된다. 그 프레임에 대해 무엇을 *말할* 수 있는지는 역할(§ 1.3)에 따라 다르다.

디바이스는 그것을 조용히 폐기하여야 하며, 응답하여서는 안 된다. 무결성 검사에 실패한 프레임의 CMD 와 SEQ 는 신뢰할 수 없으므로, 응답을 실을 오피코드도 없고 응답에 담을 토큰도 없다. 응답한다면 그것은 추측이며, 애초에 보낸 적이 없을 수도 있는 요청에 답하는 셈이 된다.

브리지는 자신이 읽은 CMD 와 SEQ 로 [ERR][ERR_BAD_CRC] 를 응답하여도 되며, 참조 브리지는 그렇게 한다. 브리지의 처지는 디바이스와 다르다. 전달하기를 거부한 프레임에 대해 아무 말도 하지 않는 브리지는, 프레임을 받고도 무시한 디바이스와 구별되지 않는다. 응답한다는 것은 자신이 볼 수 있는 링크에 그 손상을 귀속시키는

일이다. 같은 논리는 복구만을 역할로 하는 구현 — 부트로더 — 에도 이어진다. 거기서는 침묵이 애초에 기동조차 하지 않은 보드와 구별되지 않는다.

호스트는 두 결과를 모두 견뎌야 한다. 아무 응답도 없는 경우, 그리고 자신이 알지 못할 수도 있는 SEQ 를 실은 [ERR][ERR_BAD_CRC] 가 오는 경우.

세 검사를 모두 통과한 뒤에야 프레임은 수락된다. 오피코드가 무엇으로 밝혀지든, § 5.3 의 링크 활성 시계를 먹이는 것은 이 수락이다.

3.8 최대 프레임 크기

디바이스는 자신이 받아들이는, 구분자를 제외한 가장 큰 인코딩된 프레임을 문서화하여야 한다. 그 값은 제품 상수이며 Part II 에 기록된다.

이 한계는 지침이 아니라 실제 경계이다. 수신 측은 인코딩된 바이트를 고정 크기 버퍼에 모으고 그 끝을 넘어간 바이트는 버려지므로, 너무 긴 프레임은 잘린 프레임으로 도착하여 § 3.7 에 의해 폐기된다 — 조용히, 오류 응답 없이. 그런 프레임을 보낸 호스트가 관측하는 것은 아무것도 없으며, 그것은 죽은 링크에서 관측하는 것과 똑같다.

- 호스트는 디바이스가 문서화한 한계보다 긴 프레임을 보내서는 안 된다.
- 호스트는 그 한계를 실험으로 알아내려 하여서는 안 된다. 응답을 받은 프레임이 증명하는 것은 그 프레임이 들어갔다는 사실뿐이다.
- 디바이스는 자신이 구현한 명령 그룹이 요구할 수 있는 모든 프레임을 받아들여야 한다. 디바이스는 자신이 응답하는 명령을 배제하는 한계를 문서화하여서는 안 된다.

4. 요청, 응답, 오류

4.1 모델

프로토콜은 요청/응답 방식이며, 비요청 부채널을 함께 가진다.

- 디바이스가 처리하는 모든 요청은 같은 CMD 와 같은 SEQ 를 실은 응답 프레임을 **정확히 하나** 만들어 낸다.
- 디바이스는 한 요청에 두 번 응답하여서는 안 되며, 처리한 요청을 응답 없이 두어서도 안 된다 — § 5.2 의 조용한 폐기 모드 하나만이 예외이다.
- 디바이스 → 호스트 방향의 비요청 프레임(§ 7)은 SEQ = 0 을 싣고 아무것에도 답하지 않는다.
- 디바이스가 요청을 동시에 처리할 의무는 없으며, 호스트는 그렇게 한다고 가정하여서는 안 된다.

4.2 상태 바이트

응답 페이로드는 항상 [STATUS u8][DATA ...] 이다.

STATUS	의미
0x00	OK. DATA 는 그 명령의 결과이며 비어 있을 수 있다.
0x01	ERR. DATA[0] 은 오류 코드(§ 4.3)이다. 그 뒤의 DATA 바이트는 허용되며 오류 코드마다 의미가 다르다.
0x02	예약. 적합한 디바이스는 이 값을 내보내지 않는다.

호스트는 0x00 이 아닌 모든 상태를 실패로 취급하여야 하며, 알지 못하는 상태 값 때문에 동기를 잃어서는 안 된다. 프레임을 소비하고, 실패를 보고하고, 계속 진행한다. 특히 0x02 는 비어 있는 값이 아니라 예약된 값이다 — 이 프로토콜의 이전 개정판이 이 값을 두 번째 오류 상태로 선언했었다 — 따라서 아무도 보내지 않더라도 수신 측은 그것이 도착하는 상황을 견뎌야 한다.

디바이스는 [0x01] 만 보내지 말고 [0x01][오류 코드] 를 보내야 한다. 호스트는 코드 없는 오류와 잘린 오류를 구별할 수 없다.

4.3 핵심 오류 공간

오류 코드 0x00 부터 0x0F 까지가 **핵심 공간** 이다. 이 공간은 여기서만 정의되며, 정확히 다음 코드들만을 담는다.

Code	Name
0x00	ERR_OK
0x01	ERR_BAD_LEN
0x02	ERR_BAD_CRC
0x03	ERR_BAD_ARGS
0x04	ERR_NOT_READY
0x05	ERR_UNKNOWN

각 코드는 정확히 하나의 의미를 가지며, 이 본문이 그 의미이다.

- **ERR_OK (0x00)** — 오류 없음. 코드 공간에서의 0 이 상태 바이트에서의 0 과 같은 뜻이 되도록 존재하며, **절대 전송되지 않는다**. 디바이스는 성공을 상태 0x00 으로 보고하며 오류 코드는 아예 실지 않는다. [0x01][0x00] — 오류 상태가 ERR_OK 를 싣고 있는 것 — 은 형식에 어긋난 응답이며, 호스트는 이를 성공이 아니라 실패로 취급하여야 한다.
- **ERR_BAD_LEN (0x01)** — 페이로드가 그 명령이 요구하는 길이보다 짧았다. 페이로드 안의 어떤 값도 살펴 보기 전에 판정된다.
- **ERR_BAD_CRC (0x02)** — 어떤 프레임이 무결성 검사에 실패했고, 그것을 받은 계층이 조용히 폐기하는 대신 그 사실을 보고하고 있다. 디바이스는 이 코드를 보내서는 안 되며, 브리지나 부트로더는 보내어도 된다(§ 3.7).
- **ERR_BAD_ARGS (0x03)** — 페이로드 길이는 충분했지만 그 안의 무언가가 범위를 벗어났거나, 알 수 없는 값이거나, 쓸 수 없는 대상이었다. 디바이스가 처리하지 않는 오피코드에 대한 답이기도 하다(§ 9.1).
- **ERR_NOT_READY (0x04)** — 명령을 인식했고 인자도 유효하지만, 디바이스의 현재 상태가 그것을 허용하지 않는다. 같은 요청이 그대로 나중에 성공할 수 있다. 이미 열려 있는 세션(§ 6.4), 아직 진입하지 않은 모드, 그 밖에 호스트가 충족하지 못한 모든 전제 조건에 대한 답이다.
- **ERR_UNKNOWN (0x05)** — 오피코드를 인식하지 못했다. 선택 사항이며, 그 조건에 대해서는 ERR_BAD_ARGS 와 서로 바꾸어 쓸 수 있다. § 9.1 을 보라.

제품은 0x00-0x0F 안에 새 코드를 정의하여서는 안 되고, 기존 코드에 두 번째 의미를 부여하여서는 안 되며, 위에 정의된 조건이 아닌 다른 조건에 그 코드를 사용하여서도 안 된다. 이 코드들은 호스트가 상대가 무엇인지 몰라도 해석할 수 있는 코드이다.

가장 흔한 두 코드의 모양을 잘 보라. `ERR_BAD_LEN` 은 페이로드가 짧아서 명령을 아예 실행할 수 없었다는 뜻이며, 페이로드 안의 어떤 값도 살펴보기 전에 판정된다. `ERR_BAD_ARGS` 는 페이로드 길이는 충분했지만 그 안의 무언가가 범위를 벗어났거나, 알 수 없는 값이거나, 쓸 수 없는 대상이었다는 뜻이다. 디바이스는 짧은 페이로드에 `ERR_BAD_ARGS` 로 답하거나 잘못된 값에 `ERR_BAD_LEN` 으로 답하여서는 안 된다. 호스트는 그 차이로 잘못 만든 프레임과 잘못 고른 값을 구별한다.

명령의 페이로드 길이는 **최솟값**이며, 일치 검사를 받는 것은 `LEN` 뿐이다(§ 3.2). 디바이스는 페이로드가 명령이 요구하는 것보다 **길다**는 이유로 `ERR_BAD_LEN` 으로 답하여서는 안 된다. 명령이 정의한 바이트를 읽고 나머지는 무시하여야 한다. 호스트는 읽힐 것을 기대하며 뒤에 바이트를 덧붙여 보내서는 안 되며, OK 응답을 그것이 읽혔다는 증거로 받아들여서도 안 된다. 어느 쪽이든 같은 응답이 돌아오며, 바로 그 때문에 남은 바이트는 의미를 실을 수 없다.

4.4 확장 공간

`0x10` 이상의 코드는 **확장 공간**이며, 이 표준은 그 안에 아무것도 할당하지 않는다. 그곳의 코드는 그것을 보낸 계층이 말하는 의미를 가지며, 호스트가 어느 계층이 응답했는지 알아야만 해석할 수 있다.

이는 가정이 아니다. 브리지 계층(§ 9.2)은 SWD 및 코어 제어 동작에 `0x11-0x15` 를, 플래시 동작에 `0x20, 0x21, 0xE1` 을, 자신의 펌웨어 갱신에 `0x30-0x35` 를 쓴다. 오류 코드와 오피코드는 서로 다른 공간이다. 여기에서 이름을 든 `0xE1` 은 오류 코드이며, 같은 `0xE1` 이 § 9.2 의 브리지 예약 *오프/코드* 대역 안에도 든다는 사실은 이 코드에 대해 아무것도 말해 주지 않는다. 그 바이트가 프레임의 어디에 놓이느냐가 어느 공간에 속하는지를 정한다. 첫 블록은 정확히 짝여 둘 값어치가 있다. `0x14` 와 `0x15` 는 코어 정지와 코어 재개이며, SWD 오류가 아니면서 SWD 오류들 옆에 놓여 있다. 이를 전송 계층 결함으로 읽는 바람에 실측 시간이 낭비된 적이 있다. 그 의미들은 브리지의 것이고 — 여기 나열한 블록이 그 계층 할당의 전부이며, Part II 의 브리지 부록이 그 코드들을 일으키는 명령을 싣고 있다 — 다른 계층이 같은 번호에 무엇을 할당할지에 대해서는 아무것도 말하지 않는다. 한 링크 위의 디바이스와 브리지가 `0x30` 을 서로 무관한 용도로 함께 써도 충돌하지 않는다. 어느 쪽이 응답했는지는 프레임이 말해 주기 때문이다.

제품이 자신의 오류 코드가 필요할 때 지켜야 할 규칙은 다음과 같다.

1. `0x10` 이상에서 할당한다. 핵심 공간에는 절대 할당하지 않는다.
2. 그 할당을 소유한 계층에 대해 Part II 에 문서화한다.
3. 자기 계층 밖의 번호에 대해서는 아무것도 가정하지 않는다 — 다른 계층이 쓰지 않는다는 이유만으로 그 번호가 비어 있는 것은 아니다.

5. 동작 모드

5.1 모드

디바이스는 항상 정확히 하나의 동작 모드에 있다. 그것을 바꾸는 것은 `CMD_SET_MODE`(§ 6.1)이다.

Value	Name
0x00	OPMODE_ISP
0x01	OPMODE_NORMAL
0x02	OPMODE_DEBUG

Value	Name
0x03	OPMODE_TUNING
0x04	OPMODE_TEST

이 표준은 값을 정하고 각 모드가 무엇을 위한 것인지를 고정한다. 어떤 모드가 제품 안에서 무엇을 하는지 — 어떤 서브시스템을 시작하고, 중단하고, 무시하는지 — 는 제품이 정하며 Part II 에 기록된다.

- **OPMODE_ISP** — 제품의 인시스템 프로그래밍 경로를 선택한다. 이 모드를 구현한 디바이스는 그 모드에 있는 동안 자신의 통상적인 입력에 반응하지 않아도 되며, 더 이상 아무것도 도착하지 않으면 스스로 OPMODE_NORMAL 로 돌아가도 된다.
- **OPMODE_NORMAL** — 부팅 기본값이자 조용한 모드. § 5.2 를 보라.
- **OPMODE_DEBUG** — 디바이스가 구현한 모든 명령에 응답한다.
- **OPMODE_TUNING** — 호스트가 주도하는 교정(§ 6.4)을 위해 예약되었다.
- **OPMODE_TEST** — 제품 자신의 양산 시험을 위해 예약되었다.

디바이스는 OPMODE_NORMAL 로 부팅하여야 한다. 디바이스는 자신이 구현하지 않은 모드 값에 대해 CMD_SET_MODE 를 무시하거나 이웃한 모드를 선택하지 말고 ERR_BAD_ARGS 로 답하여야 한다.

디바이스가 받아들이기만 하고 실제로 아무 일도 하지 않는 예약 모드라도 응답은 OK 이다 — 그 응답의 뜻은 "값을 인식했다" 이지 "모드가 무언가를 했다" 가 아니다.

5.2 NORMAL 이 응답하는 것

OPMODE_NORMAL 은 부팅 기본값이며 의도적으로 조용하다. 호스트가 요청하기 전까지 디바이스 자신의 타이밍을 호스트 트래픽으로부터 격리한다.

OPMODE_NORMAL 에서 디바이스는 정확히 다음 명령들을 처리하고 응답하여야 한다. CMD_PING, CMD_INFO, CMD_GET_VERSION, CMD_SET_MODE, CMD_DBG_ONLINE, CMD_RESET(§ 6.1, § 6.2). 그 밖의 모든 오피코드는 — 할당된 것이든, 할당되지 않은 것이든, 구현되지 않은 것이든 똑같이 — **조용히 폐기하여야 한다**. 오류조차 없이 어떤 응답도 하지 않으며, 아무 동작도 하지 않는다.

이 여섯 개가 규범적 집합이며, 다른 어디에 적힌 목록도 권위를 갖지 않는다.

이곳은 이 표준에서 형식이 올바른 요청이 응답을 받지 못하는 유일한 지점이며, 양쪽 끝이 이를 같은 방식으로 이해하여야 한다. OPMODE_NORMAL 에서 응답을 받지 못한 호스트가 찾아낸 것은 고장 난 디바이스가 아니라, 아무도 부팅 모드에서 나오라고 요청하지 않은 디바이스이다. CMD_SET_MODE 는 모든 모드에서 처리되며, 거기서 빠져나오는 유일한 길이다.

CMD_PING, CMD_INFO, CMD_GET_VERSION 이 모든 모드에서 응답되는 것은 설계에 따른 것이다. 디바이스를 식별하고 살아 있는지 확인하는 일이 디바이스의 상태를 바꾸도록 요구하여서는 안 된다.

5.3 링크 활성

OPMODE_NORMAL 이 아닌 모드에 있는 동안, 링크 활성 간격 ONLINE_TIMEOUT_MS 보다 오래 어떤 프레임도 수락되지(§ 3.7) 않으면 디바이스는 스스로 OPMODE_NORMAL 로 돌아가야 한다. 이 타이머는 특정 오피코드가 아니라 수락된 모든 프레임에 의해 초기화된다.

따라서 OPMODE_NORMAL 밖에 머무르려는 호스트는 그 간격마다 *무엇이든* 보내야 한다. CMD_DBG_ONLINE 은 가장 값싼 선택지로 존재하지만 — 페이로드를 받지 않고 그 밖에 아무 일도 하지 않는다 — 수락되는 프레임이면 무엇이든 그 역할을 한다.

이 규칙의 목적은 활성 보고가 아니다. 부팅 기본값이 아닌 모드는 디바이스 자신의 보호 동작 일부를 중단시켜 두었을 수 있고, 케이블이 뽑혔다고 해서 그것이 중단된 채로 남아서는 안 된다는 것이 목적이다. 적합한 디바이스는 이 복귀를 무조건적으로 만들어야 한다. 방어하려는 실패가 바로 호스트가 사라지는 것이므로, 호스트가 무언가를 확인해 주는 것에 의존할 수 없다.

이 간격의 값은 제품 상수이며 Part II 에 보고된다.

6. 핵심 명령 집합

이 절의 표들은 권위 있는 오피코드 정의로부터 생성된다. 요청과 응답의 모양은 표에 있으며, 본문은 그 주변의 규칙을 말한다.

6.1 식별, 모드, 활성

ID	Name	Request	Response	Notes
0x01	CMD_PING	[]	[OK][<i>"PONG"</i>]	
0x02	CMD_INFO	[]	[OK][id string, contains "CFS"]	
0x03	CMD_SET_MODE	[op_mode u8]	[OK] or [ERR][ERR_BAD_ARGS]	
0x04	CMD_DBG_ONLINE	[]	[OK]	liveness ping, processed in any op_mode
0x05	CMD_GET_VERSION	[sel u8]	[OK][sel][major][minor][patch]	

이 그룹의 모든 명령은 적합한 모든 디바이스가 구현하여야 하며, 그 하나하나가 모든 동작 모드에서 응답된다.

- **CMD_PING** — 활성 확인용 프로브. 응답 페이로드는 고정된 문자열이며, 호스트는 그 문자열을 파싱하기보다 상태 바이트를 확인할 것을 권장한다.
- **CMD_INFO** — 사람이 읽는 식별 문자열. 자유 문자열이며 내용은 제품이 정한다. 호스트는 판단을 내리기 위해 이것을 파싱하여서는 안 된다. 그 용도로는 CMD_GET_VERSION 이 있다.
- **CMD_SET_MODE** — § 5 를 보라.
- **CMD_DBG_ONLINE** — § 5.3 의 활성 타이머를 초기화하는 것만이 유일한 효과인 무동작 명령.
- **CMD_GET_VERSION** — 기계가 읽는 식별 정보. 하나의 오피코드가 앞자리 선택 바이트를 통해 둘 이상의 질문에 답하며, 응답이 그 선택자를 되돌려주므로 응답 자체가 자기 서술적이다. 로그에서 읽어도 알 수 있고, 두 요청을 파이프라이닝한 호스트도 맞출 수 있다.

요청은 [sel u8], 응답은 [OK][sel u8][major u8][minor u8][patch u8] 이며, 선택자는 DATA[0] 에 되풀이된다. 이 표준은 두 개의 선택자를 정의한다.

Value	Name	Meaning
0x00	VER_SEL_FW	the firmware build on this board
0x01	VER_SEL_CMD_SET	the opcode table it implements

VER_SEL_FW 는 응답한 펌웨어 이미지의 버전을 보고한다 — 제품 고유의 식별 정보이며 제품 사이에서 비교할 수 없다. VER_SEL_CMD_SET 은 § 9.3 의 명령 집합 버전을 보고하며, 이 표준의 모든 구현이 이를 지니고 있고 모든 구현 사이에서 비교할 수 있다. 둘 다 같은 MAJOR.MINOR.PATCH 모양이며, 어느 하나에서 다른 하나를 이끌어 낼 수는 없다.

선택자 값 0x02 이상은 이 표준이 할당하지 않았다. 디바이스는 자신이 구현하지 않은 선택자에 대해 ERR_BAD_ARGS 로 답하여야 하며, 다른 선택자의 값으로 답하여서는 안 된다. 두 요청을 파이프라이닝한 호스트가 응답을 구별할 수단은 되돌아온 선택자뿐이다. 적합한 모든 디바이스는 위의 두 선택자를 모두 구현하여야 한다. 선택자 번호는 오피코드마다 따로이며 공유되지 않는다. 브리지 계층은 자신의 버전 명령 (§ 9.2, Part II)과 자신의 선택자 공간을 가지고 있고, 거기서 같은 선택자 값은 다른 것을 뜻한다.

호스트는 이 그룹 밖의 오피코드에 의존하기 전에 명령 집합 버전을 읽을 것을 권장한다.

6.2 디바이스 제어

ID	Name	Request	Response	Notes
0x10	CMD_RESET	[]	[OK][<code>"SYSTEM RESET"</code>], then <code>NVIC_SystemReset()</code>	
0x11	CMD_ACTIVATE	[]	[OK] (stub)	
0x12	CMD_DEACTIVATE	[]	[OK] (stub)	
0x13	CMD_START_HEATING	[]	[OK][3 bytes] (stub)	
0x14	CMD_STOP_HEATING	[]	[OK][3 bytes] (stub)	

이 그룹에서 모든 디바이스에 요구되는 명령은 CMD_RESET 이며, 모든 모드에서 응답된다. 디바이스는 리셋하기 **전에** 응답을 보내야 하고, 그 응답이 먼저 송신 경로를 빠져나가도록 허용하여야 한다. 리셋 뒤에는 어떤 프레임도 오지 않는다. 호스트는 그 이후의 링크를 다시 초기화된 것으로 취급하여야 한다 — 디바이스는 OPMODE_NORMAL (§ 5.1)로 돌아온다.

이 그룹의 나머지 오피코드는 제품 자신의 활성화 제어와 가열 제어를 놓는 표준화된 자리이다. 이 중 하나를 구현하지 않은 제품은 그것을 확인 응답하지 말고 § 9.1 에 따라 답하여야 하며, 이 오피코드들을 무관한 기능에 재사용하여서는 안 된다.

알려진 이탈이며, 덮지 않고 밝힌다. 참조 펌웨어는 오늘 그 규칙을 지키지 않는다. CMD_ACTIVATE, CMD_DEACTIVATE, CMD_START_HEATING, CMD_STOP_HEATING 은 제품이 실제로 그에 반응하든 아니든 [OK] 로 답하며, 위 표가 이 넷을 각각 스텝으로 표시하는 이유가 그것이다. 규칙은 그대로 유효하고 새 구현은 이 동작을 따라 하여서는 안 된다. 이 상태가 지속되는 동안의 귀결은, 그 네 오피코드에서 온 [OK] 는 무언가 일어났다는 증거가 아니며 호스트가 그것을 증거로 읽어서는 안 된다는 것이다. 어떤 제품이 여기 해당하는지는 Part II 가 기록한다.

6.3 레지스터 접근

ID	Name	Request	Response	Notes
0x20	CMD_REG_WRITE8	[reg_addr u8][reg_data u8]	[OK][resp_data u8]	
0x21	CMD_REG_READ8	[reg_addr u8]	[OK][resp_data u8]	
0x22	CMD_REG_WRITE16	[reg_addr u8][reg_data u16]	[OK][resp_data u16]	

ID	Name	Request	Response	Notes
0x23	CMD_REG_READ16	[reg_addr u8]	[OK][resp_data u16]	
0x24	CMD_REG_BURST_READ16	[reg_addr u8]	[OK][resp_data u16]	

주변 장치나 동반 칩의 레지스터 파일을 노출하는 제품을 위한 선택적 그룹. 레지스터 주소와 그 내용은 전적으로 제품이 정한다. 레지스터 파일이 없는 제품은 이 그룹에 § 9.1 에 따라 답하여야 한다.

6.4 튜닝

ID	Name	Request	Response	Notes
0x70	CMD_TUNING_START	[tuning_id u8][duration_ms u32][log_fields_mask u16] [report_period_ms u16]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x71	CMD_TUNING_REPORT_FRAME		[OK][tick_ms u32][field u16]... (one u16 per set bit in log_fields_mask, LSB first)	(fw->host only, seq=0) same shape family as CMD_LOG_BURST_FRAME
0x72	CMD_TUNING_END	[]	[OK]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0), same idempotent shape as CMD_LOG_BURST_STOP
0x73	CMD_SET_TUNING_PARAMETER	[tuning_id u8][coil_idx u8] [heating_mode u8] [heater_type u8][step u8] [value u16 LE]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required
0x74	CMD_GET_TUNING_PARAMETER	[tuning_id u8][coil_idx u8] [heating_mode u8] [heater_type u8][step u8]	[OK][value u16 LE] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required

호스트가 주도하는 교정을 위한 선택적 그룹으로, 디바이스가 자신의 실제 양산 제어 알고리즘을 호스트가 제어하는 타이밍 아래에서 실행하여 실측 장비가 그것을 측정할 수 있게 한다. 이 표준은 내용이 아니라 **모양** 을 고정한다.

- CMD_TUNING_START 는 세션을 연다. 세션이 활성화인 동안 들어온 두 번째 요청은 ERR_NOT_READY 로 거절하여야 한다.
- 세션이 활성화인 동안 디바이스는 요청받은 주기로, seq = 0 을 실어, § 7.2 의 필드 배치로 보고 프레임을 비요청 전송한다.
- CMD_TUNING_END 는 세션을 닫으며 CMD_LOG_BURST_STOP(§ 7.2)과 똑같이 **이중 용도** 이다. 호스트가 시작한 경우에는 요청의 seq 를 되돌려주며 멎는다. 디바이스가 시작한 경우에는 — 세션의 지속 시간이 다 되었거나 디바이스가 스스로 실행을 끝냈기 때문에 — seq = 0 으로 도착한다. 호스트는 두 도착 경로를 같은 "세션 종료" 사건으로 처리하여야 한다.
- 디바이스는 자신의 사정으로 세션을 일찍 끝내어도 된다. 호스트는 세션이 요청한 지속 시간을 다 채운다고 가정하여서는 안 되며, 모든 세션이 적어도 한 개의 보고 프레임을 낸다고 가정하여서도 안 된다.
- 이 그룹의 파라미터 명령은 인덱스의 조합으로 교정 표를 지정한다. 모든 인덱스는 자신이 지정하는 표의 실제 차원에 대해 범위 검사를 하여야 하며, 위반은 ERR_BAD_ARGS 여야 한다 — 절대 클램프해서도, 이웃 칸에 조용히 써서도 안 된다.

어떤 튜닝 식별자가 존재하는지, 각각이 무엇을 훑는지, 각각에서 인덱스 조합이 무엇을 뜻하는지, 그리고 세션이 제품 자신의 어떤 보호 동작 중단을 수반하는지는 모두 제품이 정하며 Part II 에 기록된다.

6.5 디스플레이

ID	Name	Request	Response	Notes
0x60	CMD_DISP_ON	[]	[OK]	
0x61	CMD_DISP_OFF	[]	[OK]	

선택적 그룹. 호스트가 제어할 수 있는 디스플레이가 없는 제품은 § 9.1 에 따라 답한다.

7. 로그와 텔레메트리 스트리밍

이 절의 두 메커니즘은 모두 어떤 요청에도 답하지 않는 디바이스 → 호스트 방향 프레임을 보낸다. 그 전부가 SEQ = 0 (§ 3.5)을 싣는다.

두 그룹 모두 **선택 사항** 이다. 둘 다 구현하지 않은 제품은 그 오피코드들에 § 9.1 에 따라 답하여야 하며, 이 표준의 다른 어떤 것도 이들에 의존하지 않는다.

비요청 프레임은 응답과 정확히 같은 모양이다. 어떤 요청에도 답하지 않는다고 해서 페이로드가 달라지지 않는다. § 4.2 가 요구하는 대로 [STATUS u8][DATA ...] 이며 STATUS = 0x00 이고, 프레임 자신의 내용은 DATA[0] 에서 — 즉 페이로드 0 번째 바이트가 아니라 1 번째 바이트에서 — 시작한다. 이는 이 절의 모든 프레임과 § 6.4 의 튜닝 보고 프레임에 그대로 적용된다. 페이로드 첫 바이트를 내용으로 읽는 호스트는 자신이 취하는 모든 표본에서 한 바이트씩 어긋나며, 그 오류는 조용하다. 상태 바이트 0x00 은 첫 데이터 바이트로도 그럴듯하기 때문이다. 생성된 표본들은 그 모양을 [OK] 를 앞에 두고 명시한다.

7.1 텍스트 로그

ID	Name	Request	Response	Notes
0x30	CMD_LOG_FRAME		[OK][text bytes]	(fw->host only)
0x31	CMD_LOG_START	[]	[OK]	
0x32	CMD_LOG_STOP	[] (host) or (fw->host, unsolicited)	[OK]	

자유 형식 텍스트 진단을 위한 풀(pull) 방식 핸드셰이크로, 디바이스의 로그 출력이 링크를 넘치게 할 수 없도록 짜여 있다.

1. 호스트가 CMD_LOG_START 를 보낸다.
2. 디바이스는 로깅을 무장(arm)하기 **전에** 확인 응답을 보내야 하며, 그 뒤에 오는 무엇보다도 확인 응답을 먼저 큐에 넣어야 한다. 그래야 호스트가 어떤 로그 내용보다 확인 응답을 먼저 본다. 이 순서는 그 자체로 기능을 짊어진다. 먼저 무장하는 디바이스는 호스트가 아직 자신의 응답을 기다리는 동안 로그 한 줄을 내보낼 수 있고, 호스트는 그 줄을 자신이 요청한 응답으로 파싱하게 된다.
3. 디바이스는 다음 로그 한 줄을 텍스트를 실은 CMD_LOG_FRAME 하나로 보내고, 이어서 CMD_LOG_STOP 하나를 보낸다.
4. 호스트는 다음 줄을 위해 CMD_LOG_START 를 다시 발행한다.

호스트는 앞에 CMD_LOG_FRAME 없이 CMD_LOG_STOP 이 도착하는 것을 견뎌야 한다. 그것은 디바이스가 할 말이 없었다는 뜻이다. 호스트는 텍스트가 NUL 로 끝난다고 가정하여서는 안 되며, 페이로드 길이를 권위 있는 것으로 삼아야 한다.

호스트는 또한 CMD_LOG_START 를 더 받지 않고도 계속 프레임을 내보내는 디바이스를 견뎌야 한다. 스트림을 한 줄로 한정하는 것은 무장 해제 단계이며, 제품은 연속 텔레메트리를 위해 의도적으로 무장을 유지하여도 된다. 스트림을 멈추어야 하는 호스트는 무장을 해제할 수단을 갖추어야 한다. OPMODE_NORMAL 을 선택하는 CMD_SET_MODE 는 언제나 그 역할을 하며, 제품은 로깅을 파라미터 슬롯(§ 8)으로 노출하여도 된다.

텍스트 로깅은 진단용이다. 고정된 주기로 표본화되는 것은 § 7.2 에 속한다 — 그런 속도에서의 텍스트 서식화 와 파싱이야말로 이진 경로가 피하려고 존재하는 비용이다.

7.2 이진 텔레메트리 버스트

ID	Name	Request	Response	Notes
0x40	CMD_LOG_BURST_START	[fields_mask u16] [period_ms u16] [duration_ms u32]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x41	CMD_LOG_BURST_FRAME		[OK][tick_ms u32][field u16]... (one u16 per set bit in fields_mask, LSB first)	(fw->host only, seq=0)
0x42	CMD_LOG_BURST_STOP	[]	[OK][total_frame_cnt u16]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0)

시계열 측정을 위한 고정 폭 이진 스트림.

세션 시작. CMD_LOG_BURST_START 는 필드 비트마스크, 보고 주기, 지속 시간을 받는다. 주기는 [BURST_PERIOD_MS_MIN, BURST_PERIOD_MS_MAX] 안에 있어야 하고, 지속 시간은 0 이 아니라면 [BURST_DURATION_MS_MIN, BURST_DURATION_MS_MAX] 안에 있어야 한다. 어느 범위든 벗어난 값은 클램프하지 말고 ERR_BAD_ARGS 로 거절하여야 한다.

이들은 이 표준의 프로토콜 상수이며 모든 제품에서 동일하고, 그 값이 사는 곳은 여기이다.

Name	Value
BURST_DURATION_MS_INFINITE	0
BURST_DURATION_MS_MAX	1000000
BURST_DURATION_MS_MIN	100
BURST_MAX_FIELDS	16
BURST_PERIOD_MS_MAX	1000
BURST_PERIOD_MS_MIN	10

BURST_DURATION_MS_INFINITE 는 예약된 지속 시간 값이며 지속 시간 범위의 구성원이 아니다. 지속 시간이 없다는 뜻이며, 이 값으로 시작한 세션은 호스트가 멈출 때까지(아래 **정지**) 계속된다. BURST_DURATION_MS_MIN 보다 작으면서도 디바이스가 반드시 받아들여야 하는 유일한 지속 시간 값이다. BURST_MAX_FIELDS 는 마스크가 실을 수 있는 필드 비트의 개수이며, 마스크가 u16 인 이유가 그것이다.

주기와 지속 시간은 모두 밀리초 단위이다.

필드 마스크 는 표준 LOG_FIELD_* 계열에서 신호를 비트 하나씩 선택한다. 비트 할당은 이 표준의 일부이다.

Bit	Name
0x01	LOG_FIELD_VDD
0x02	LOG_FIELD_VAT
0x04	LOG_FIELD_IAT
0x08	LOG_FIELD_PWR
0x10	LOG_FIELD_DUTY
0x20	LOG_FIELD_PROT
0x40	LOG_FIELD_RAT

각 비트는 하나의 양을 가리키며, 이 표준이 고정하는 것은 그 양이다.

- LOG_FIELD_VDD — 디바이스 자신의 공급 전압 레일.
- LOG_FIELD_VAT — 구동되는 부하 양단의 전압.
- LOG_FIELD_IAT — 그 부하를 흐르는 전류.
- LOG_FIELD_PWR — 그 부하에 전달되는 전력.
- LOG_FIELD_DUTY — 구동의 듀티 사이클.
- LOG_FIELD_PROT — 디바이스의 보호 상태. 비트필드이다.
- LOG_FIELD_RAT — 디바이스가 계산한 부하 저항.

모든 필드는 u16 으로 실린다. 두 가지는 제품이 정하며 Part II 에 기록된다. 각 필드 뒤에 있는 **스케일, 단위, 산출 방식** — 제품의 스케일을 모른 채 값을 읽은 호스트가 손에 쥔 것은 측정값이 아니라 숫자일 뿐이다 — 그리고 제품이 실제 측정에 **어떤 비트를 연결해 두었는지**.

디바이스는 자신이 연결하지 않은 비트도 오류 없이 받아들여야 하며, 그 필드를 0 으로 보고하여야 한다. 가장 높이 할당된 비트보다 위의 비트는 이 표준이 할당하지 않았으며, 디바이스는 할당되지 않은 비트를 연결되지 않은 비트와 정확히 똑같이 다루어야 한다. 받아들이고, 0 으로 보고한다. 두 규칙 모두 나중 제품에 맞춰 만든 호스트가 더 오래된 제품에서 실패하지 않도록 존재한다.

프레임 배치. 각 CMD_LOG_BURST_FRAME 의 페이로드는 [STATUS][tick_ms u32][field u16] ... 이다.

§ 4.2 의 상태 바이트로 항상 0x00, 그 다음 밀리초 단위 u32 타임스탬프, 그 다음 **요청된 마스크에서 세워진** 비트마다 u16 하나씩을 비트 오름차순으로 싣는다. 순서는 비트 번호로 정해지며 호스트가 고르는 것이 아니다. 세워지지 않은 비트는 u16 을 전혀 싣지 않는다. 타임스탬프는 한 세션 안에서 감소하여서는 안 되며 그 0 점이 세션 도중에 바뀌어서도 안 된다. 그 0 점이 어디인지는 제품이 정한다. 마스크는 프레임 안에 되풀이되지 않는다. 호스트는 자신이 보낸 마스크로 디코딩하여야 한다. 디바이스는 값의 순서를 바꾸어서는 안 되며, 하나를 빠뜨려서도 안 된다.

정지. CMD_LOG_BURST_STOP 은 이중 용도이며, 튜닝 그룹이 재사용하는 모양이 바로 이것이다(§ 6.4).

- **호스트가 시작** — 요청이다. 디바이스는 활성 세션을 정지하고 그 요청 자신의 seq 로 응답한다. 이 명령은 멍든이어야 한다. 동작 중이 아닌 세션을 정지하는 것은 오류가 아니라 OK 이다. 무한 지속 세션을 끝내는 유일한 방법이며, 유한한 세션을 일찍 취소하기도 한다.

- *디바이스가 시작* — 비요청, SEQ = 0, 0 이 아닌 지속 시간이 다 되었을 때 보낸다.

둘은 같은 페이로드를 싣는다. 호스트는 어느 쪽으로든 자신의 세션 상태를 확정하여야 한다.

동시성. 표본화가 명령 디스패치를 막아서는 안 된다. 세션이 활성인 동안에도 호스트 자신의 정지 요청을 포함한 통상 트래픽이 계속 흘러야 한다. 명령 처리기 안에서 스왑을 실행하는 디바이스는 정지시킬 수 없으며, 적합하지 않다.

8. 변수와 파라미터

ID	Name	Request	Response	Notes
0x50	CMD_VAR8_SET	[var_id u8][value u8]	[OK]	
0x51	CMD_VAR8_GET	[var_id u8]	[OK][value u8]	
0x52	CMD_VAR16_SET	[var_id u8][value u16 LE]	[OK]	
0x53	CMD_VAR16_GET	[var_id u8]	[OK][value u16 LE]	
0x54	CMD_VAR32_SET	[var_id u8][value u32 LE]	[OK]	
0x55	CMD_VAR32_GET	[var_id u8]	[OK][value u32 LE]	
0x56	CMD_PAR8_SET	[par_id u8][value u8]	[OK]	
0x57	CMD_PAR8_GET	[par_id u8]	[OK][value u8]	
0x58	CMD_PAR16_SET	[par_id u8][value u16 LE]	[OK]	
0x59	CMD_PAR16_GET	[par_id u8]	[OK][value u16 LE]	
0x5A	CMD_PAR32_SET	[par_id u8][value u32 LE]	[OK]	
0x5B	CMD_PAR32_GET	[par_id u8]	[OK][value u32 LE]	

두 계열의 명령이 디바이스 상태에 인덱스로 접근하게 해 주며, 각각 세 가지 폭을 가진다. 둘을 가르는 것은 무엇에 닿는가이다.

- **CMD_VAR*** — **변수** 맵. 살아 있는 런타임 상태이다. 대체로 펌웨어가 실제로 하고 있는 일(상태, 측정값, 타이머, 게이지)에 대한 읽기 전용 시야이며 그것의 두 번째 사본이 아니다. 여기에 더해, 디바이스가 원래라면 자신의 하드웨어에서 읽었을 것을 호스트가 대신 공급하는 입력으로 이루어진 작은 쓰기 가능 그룹이 있다.
- **CMD_PAR*** — **파라미터** 맵. 디바이스의 설정과 임계값 저장소이다.

이 그룹은 **선택 사항** 이다. 두 계열을 모두 구현하지 않은 제품은 열두 개 오피코드 전부에 § 9.1 에 따라 답하여야 한다. 제품은 한 계열만 구현하여도 되고 한 폭만 구현하여도 되지만, 구현한 폭은 그 SET 과 GET 을 모두 구현하여야 한다. 쓸 수는 있는데 다시 읽을 수 없는 값은 확인할 방법이 없기 때문이다.

두 계열은 하나의 모양을 공유한다. SET 은 [id u8][value] 를 받고, GET 은 [id u8] 을 받아 [OK][value] 로 답하며, 값은 명령의 폭에 따라 u8, u16, u32 리틀 엔디언이다. 세 폭은 서로 별개인 세 개의 식별자 공간이다. 8 비트 공간의 id 3 과 16비트 공간의 id 3 은 서로 무관한 슬롯이다.

요구되는 동작:

- 그 폭의 표 크기와 같거나 그보다 큰 id 는 어떤 디스패치도 시도하기 전에 ERR_BAD_ARGS 로 답하여야 한다.
- 읽기 전용 슬롯에 대한 SET 은 ERR_BAD_ARGS 로 답하여야 한다. 값을 저장하여서는 안 되고 OK 로 답하여서도 안 된다. 호스트는 조용히 무시된 쓰기와 성공한 쓰기를 구별할 수 없다.
- 적합한 **디바이스** 는 쓰기 가능한 각 슬롯에 대해 자신이 받아들이는 범위를 문서화하여야 한다. 그 범위는 디바이스가 자기 자신에 대해 하는 진술이다. 폭이 담을 수 있는 모든 값을 받아들이는 슬롯은 그렇다고 문서화하고, 그보다 적게 받아들이는 슬롯은 자신이 받아들이는 것을 문서화한다. 이 표준은 그 문서가 어디에 살아야 하는지도, 어떤 형태여야 하는지도 규정하지 않는다 — 데이터시트든, 제품 설명서든, Part II 가 쓰는 형태의 부록이든 상관없다 — 다만 디바이스 자신의 범위 진술이 존재하고 호스트가 쓰는 것이 그 것이어야 한다는 점만을 규정한다. 호스트 도구가 자기 사본의 식별자 맵에 들고 있는 한계는 그 도구의 입력 편의이며 그것을 운용하는 사람이 조정할 수 있는 것이지 디바이스의 범위가 아니다. 호스트는 그것을 디바이스의 범위로 취급하여서는 안 된다.
- 슬롯이 받아들이지 않는 값을 실은 SET 은 ERR_BAD_ARGS 로 답하여야 한다. 디바이스는 값을 마스킹하거나 범위 안으로 클램프하여서는 안 되며, 나중에 버릴 값에 OK 로 답하여서도 안 된다. 불가능한 것을 요구한 호스트에는 버그가 있는 것이고, 방금 말한 세 가지 결과는 하나같이 그 버그를 드러낼 오류 대신 그럴듯해 보이는 측정값을 호스트의 손에 쥐여 준다.
- 같은 슬롯에 대한 SET 이 성공한 뒤에 발행된 그 슬롯의 GET 은, 그 사이에 디바이스 자신이 값을 바꾸지 않았다면, 설정된 값을 돌려주어야 한다. 이것이 이 규칙의 관측 가능한 형태이며, 호스트가 확인할 수 있는 유일한 형태이다. 값을 어디에 보관하는지는 디바이스의 사정이지만, 다시 읽을 수 없는 쓰기는 버려진 쓰기와 구별되지 않는다. 이를 충족할 수 없는 슬롯은 SET 을 ERR_BAD_ARGS 로 거절하여야 하고 읽기 전용으로 선언하여야 한다.

마지막 두 항목은 선 위에서 본 하나의 규칙이며, 호스트가 관측할 수 있는 것의 전부이다. **OK 로 답한 SET 뒤의 그 슬롯 GET 은 설정된 값을 돌려준다. 그리고 어떤 값을 받아들이지 않을 디바이스는 OK 로 답하는 대신 그렇다고 말한다.** 적합한 세 번째 결과는 없다 — 디바이스가 다른 것을 보관했다는 뜻의 OK 도, 아무것도 보관하지 않았다는 뜻의 OK 도 없다.

이 규칙이 금지하는 실패는 실수로 만들어 내기 쉬우며, 애초에 프로토콜의 실수가 아니라는 점에서 짙어 돌 값 어치가 있다. 어떤 슬롯의 설정 함수가 디바이스 내부에서 값을 거부하면서도 그 거부를 명령 처리기에 되돌려 보낼 방법이 없을 때 — void 를 반환하는 설정 함수가 흔한 모양이다 — 쓰기는 버려지고, 보고할 것이 없는 처리기는 OK 로 답한다. GET 이 마지막으로 쓴 값과 어긋나기 전까지는 선 위의 어떤 것도 그것을 반박하지 않으며, 그 슬롯을 다시 읽어 보지 않는 호스트는 아예 알지 못한다. 거부는 응답을 구성하는 계층까지 도달하여야 하고, 그 계층이 ERR_BAD_ARGS 를 보내야 한다.

8.1 식별자 맵은 제품이 정하며, 이것이 기능을 짊어진다

이 표준은 명령과 폭과 오류 동작을 정의한다. 어떤 슬롯 번호가 어떤 양을 뜻하는지는 정의하지 **않는다**. 그 맵은 제품마다 배정되며 Part II 에 기록된다.

그 맵은 제품 사이에서 **이식 가능하지 않으며**, 잘못된 맵을 쓰는 것을 프로토콜이 막을 방법은 없다. 한 제품에서 유효한 슬롯 번호는 다른 제품에서도 유효할 가능성이 매우 높고, 거기서는 완전히 다른 필드를 가리킨다 — 그래서 디바이스는 잘못된 쓰기를 수행하고 **OK** 로 답한다. 어떤 계층에서도 관측할 오류가 없다. 따라서,

- 호스트는 어떤 VAR/PAR 접근이든 발행하기 전에 자신이 어떤 제품, 어떤 명령 집합 버전과 이야기하고 있는지 확립하여야 하며(CMD_INFO, CMD_GET_VERSION), 그 제품 자신의 맵을 사용하여야 한다.

- 제품은 새 슬롯을 뒤에 덧붙이는 방식으로 배정하여야 한다. 기존 슬롯의 번호를 다시 매기면 바뀌지 않은 호스트가 쓰는 대상이 달라지며, 이는 MAJOR 명령 집합 변경(§ 9.3) 이다.

8.2 두 오피코드가 한 필드에 닿는 곳에서 둘은 일치하여야 한다

이 표준은 디바이스의 설정 가능한 상태로 가는 경로를 하나 이상 정의한다. 위의 파라미터 계열, 그리고 튜닝 그룹을 구현한 제품에서는 그 그룹 자신의 파라미터 명령(§ 6.4). 제품은 같은 하부 필드를 이들 중 둘 이상을 통해 노출하여도 된다.

그렇게 하는 경우, 그 오피코드들은 그 필드에 대해 일치하여야 한다. 그 필드가 애초에 쓰기 가능한지에 대해 일치하여야 하고, 그 필드에 대해 같은 값을 받아들여야 한다.

이 규칙이 없으면 제품은 하나의 빌드 안에서 한 필드에 대해 서로 다른 두 가지 답을 할 수 있다. 한 오피코드를 통해서 쓰기 받아들이어져 OK 로 답하고, 다른 오피코드를 통해서 동일한 쓰기가 ERR_BAD_ARGS 로 거절되는 것이다. 호스트에는 그 모순을 풀 방법이 없다. 각각의 처리기는 상대가 모르는 규칙을 적용하고 있을 뿐이라 어느 답도 그 자체로는 틀리지 않았고, 프레임 안의 어떤 것도 어느 쪽이 디바이스를 서술하는지 말해 주지 않는다. 결국 호스트는 세 번째 경로로 다시 읽어 보고 그 필드가 값을 받았는지를 판단해야 하는 처지에 놓인다.

쓰기 가능성은 필드의 속성이지 호스트가 어쩌다 그 필드에 닿은 오피코드의 속성이 아니며, 호스트는 그것을 한번 확립해 어디서나 그에 의존할 자격이 있다. 어떤 필드가 오직 하나의 오피코드로만 닿기를 의도하는 제품은, 그 필드를 다른 쪽에 매핑해 두고 거기서 거절하는 방식이 아니라 애초에 매핑하지 않는 방식으로 그 의도를 표현한다.

9. 오피코드 공간, 예약, 진화

9.1 할당되지 않은 오피코드와 구현되지 않은 오피코드

OPMODE_NORMAL 밖에서 디바이스는 자신이 처리하지 않는 **모든** 오피코드에 [ERR][ERR_BAD_ARGS] 로 답하여야 한다. 이는 두 경우를 똑같이 포괄한다. 이 표준이 할당하지 않은 오피코드, 그리고 표준이 할당했지만 이 제품이 구현하지 않은 오피코드. 침묵은 적합한 답이 아니다 — 호스트는 지원되지 않는 명령과 죽은 링크를 구별할 수 있어야 한다.

유일한 예외는 OPMODE_NORMAL (§ 5.2)의 조용한 폐기 동작이다.

ERR_UNKNOWN 은 이 목적을 위해 핵심 오류 공간에 존재하며, 인식하지 못한 오피코드와 잘못된 인자를 구별할 수 있는 디바이스는 그것을 사용하여도 된다. 호스트는 두 코드 모두 "이 디바이스는 그것을 하지 않는다" 로 받아들이어야 한다.

9.2 대역 예약

오피코드 바이트는 두 개의 대역(band)으로 나뉘며, 하나의 물리 링크가 두 개의 대화를 실행해 주는 것이 이 분할이다.

대역	소유자
0x00-0xBF	디바이스(제품 펌웨어). 새 명령은 이 안에서 위로 자란다.
0xC0-0xFF	브리지 계층에 예약됨. 제품은 여기에 명령을 정의하여서는 안 된다.

브리지는 오피코드 바이트만으로 경로를 정한다. 0xc0 이상이면 프레임에 스스로 응답하고, 0xc0 미만이면 원래의 인코딩된 바이트를 손대지 않고 디바이스로 전달한다. 따라서 0xc0 이상에 정의된 제품 명령은 브리지가 끼어 있을 때마다 도달할 수 없게 되며, 그 실패는 조용하다. 관측할 오류가 없다. 브리지가 프레임을 소비하고 스스로 응답하며, 디바이스는 그것을 보지도 못하고, 호스트는 엉뚱한 상대방으로부터 그럴듯한 응답을 받는다.

이 예약은 보드 하나의 대역이 아니라 계층 전체에 대한 하나의 대역이다. 모든 브리지 구현이 같은 표를 선언한다.

디바이스 자신의 대역 안에서 오피코드는 그룹으로 묶이며, 제품은 새 명령을 그 기능에 맞는 그룹에 놓을 것을 권장한다.

기준	그룹
0x00	식별, 모드, 활성 (§ 6.1)
0x10	디바이스 제어 (§ 6.2)
0x20	레지스터 접근 (§ 6.3)
0x30	텍스트 로그 (§ 7.1)
0x40	이진 텔레메트리 (§ 7.2)
0x50	변수와 파라미터 (§ 8)
0x60	디스플레이 (§ 6.5)
0x70	튜닝 (§ 6.4)

9.3 버전

명령 집합 버전 은 이 표준의 오피코드 맵 — 두 대역 전체 — 을 가리키는 MAJOR.MINOR.PATCH 세 쌍이다. 어떤 펌웨어의 빌드 번호도 아니며, 그것과는 독립적으로 움직인다.

- **MAJOR** 는 기존 오피코드가 번호나 의미를 바꿀 때 올라간다. 그렇지 않으면 예전 표에 맞춰 만든 호스트가 다른 처리기에 닿아 그럴듯한 답을 받게 되는데, 그것이 바로 이 버전이 검출 가능하게 만들려는 실패이다.
- **MINOR** 는 오피코드가 추가되고 기존의 모든 오피코드가 하던 뜻을 그대로 유지할 때 올라간다.
- **PATCH** 는 선 위에 영향이 없는 변경을 위한 것이다.

이 번호를 쓰는 규칙이 두 개이며, 둘을 뒤섞는 것이 멸절된 시스템이 결함도 아닌 차이 때문에 거부당하는 경로이다.

릴리스 장부 — 빌드의 속성. 한 구현의 한 릴리스 안에서 명령 집합 버전 상수의 모든 사본은 같은 값을 지녀야 하며, 값이 바뀔 때 모든 사본이 함께 움직여야 한다. 이 번호는 둘 이상의 저장소에 걸쳐 펌웨어와 호스트 도구에 중복되어 있으므로, 그 사본은 가정하지 말고 세어야 한다. Part II 가 자신이 다루는 제품에 대해 그 목록을 싣는다. 자기 자신의 명령 집합 버전을 두 가지 값으로 출하하는 릴리스는 잘못 빌드된 것이다. 이 규칙 중 선 위에서 관측 가능한 것은 아무것도 없다.

선 위 호환성 — 링크의 속성. 한 링크 위의 두 구현이 서로 다른 명령 집합 버전을 보고하여도 되며, 차이 자체는 결함이 아니다. 더 낮은 버전을 보고하는 디바이스는 자신이 어떤 오피코드 표에 맞춰 만들어졌는지를 솔직히 말하고 있는 것이고, 그것이야말로 알아 둘 값어치가 있는 사실이다. 호스트는,

- § 6.1 밖의 오피코드에 의존하기 전에 상대방의 버전(VER_SEL_CMD_SET 을 실은 CMD_GET_VERSION, § 6.1)을 읽을 것을 권장한다.
- MAJOR 차이에서는 진행을 거부할 것을 권장한다. 그 아래에서 오피코드의 번호나 의미가 바뀌었을 수 있고 그 답들은 그럴듯해 보일 것이기 때문이다.
- MINOR 나 PATCH 차이에서는 진행하여도 되며, 그 경우 두 버전 중 더 낮은 쪽이 정의하는 오피코드로 스스로를 한정할 것을 권장한다. MINOR 는 정의상 덧붙이기이므로, 더 오래된 쪽은 명령이 더 적은 것이 지 다른 것이 아니다.

10. 적합성 점검표

적합한 **디바이스** 는 아래 항목 중 자신에게 해당하는 모든 항목에 예라고 답한다.

항목이 해당하지 않는 경우. 다섯 개의 명령 그룹이 선택 사항이다. 레지스터 접근 (§ 6.3), 튜닝 (§ 6.4), 디스플레이 (§ 6.5), 스트리밍 (§ 7), 변수와 파라미터 (§ 8). 디바이스가 구현하지 않은 그룹을 시험하는 항목은 **해당 없음** 으로 답하며, 해당 없음 항목이 있는 디바이스도 여전히 적합하다 — 단 그 그룹의 오피코드에 대해 항목 25 가 성립하여야 한다. 그것이 "구현하지 않았다" 를 호스트가 들어서 아는 것이 아니라 관측할 수 있는 것으로만 들어 준다. 전송과 프레임링, 요청과 응답, 모드 아래의 어떤 항목도, 그리고 항목 22, 23, 24, 25, 26, 27 도 결코 해당 없음이 될 수 없다. 그것들은 모든 디바이스에 요구된다.

전송과 프레임링

항목	요구사항	참조
1	시리얼 링크는 합의된 속도에서 데이터 8비트, 패리티 없음, 정지 비트 1, 흐름 제어 없음이다.	2
2	호스트에게 프레임 간 간격, 브레이크, 프리앰블을 요구하지 않으며, 연달아 오는 프레임을 받아들인다.	2
3	모든 프레임은 COBS 로 인코딩되고 뒤에 0x00 구분자가 붙으며, 수신 시 0x00 은 무조건적인 프레임 경계로 취급된다.	3.4
4	인코딩 시 0 이 없는 254 바이트 연속이 완성될 때마다 0xFF 코드로 내보내고, 없던 자리에 0x00 을 만들어 넣지 않으며, 마지막 그룹은 데이터가 없더라도 코드 바이트를 싣는다.	3.4.1
5	디코딩 시 코드가 0xFF 가 아니었던 모든 그룹 뒤에 0x00 을 복원하고, 그 밖의 어떤 그룹 뒤에도 — 프레임의 마지막 그룹을 포함하여 — 복원하지 않는다.	3.4.2
6	LEN 은 CMD + SEQ + 페이로드 + CRC 를 세며, 디코딩된 길이와 일치 검사를 받는다.	3.2
7	CRC 는 CRC-16/CCITT-FALSE 이며 "123456789" 에 대해 0x29B1 을 재현한다.	3.3
8	CRC 는 디코딩된 프레임의 첫 LEN 바이트부터 마지막 페이로드 바이트까지를 덮고, CRC 바이트 자신은 제외한다.	3.3
9	COBS 디코딩, CRC 검사, LEN 검사 중 하나라도 실패한 프레임은 응답 없이 조용히 폐기된다.	3.7
10	받아들이는 가장 큰 인코딩된 프레임이 문서화되어 있으며, 그보다 긴 프레임은 검증할 수 없는 다른 프레임과 마찬가지로 조용히 폐기된다.	3.8

요청과 응답

항목	요구사항	참조
11	처리된 각 요청은 그 CMD 와 SEQ 를 되돌려주는 응답을 정확히 하나 만들어 낸다.	4.1
12	비요청 프레임은 SEQ = 0 을 싣는다.	3.5

항목	요구사항	참조
13	디바이스는 SEQ 를 결코 해석하거나 검증하지 않는다.	3.5
14	모든 응답 페이로드는 상태 바이트로 시작하며, 오류 응답은 DATA[0] 에 오류 코드를 싣는다.	4.2
15	각 핵심 오류 코드는 § 4.3 이 그것에 대해 정의한 조건에만 쓰이고, 제품 고유 코드는 0x10 이상에 있다.	4.3, 4.4
16	ERR_BAD_LEN 은 짧은 페이로드에, ERR_BAD_ARGS 는 잘못된 값에 쓰이며, 서로 바꾸어 쓰지 않는다.	4.3
17	명령이 요구하는 것보다 긴 페이로드는 받아들여 남는 부분을 무시하며, 결코 ERR_BAD_LEN 으로 답하지 않는다.	4.3

모드

항목	요구사항	참조
18	디바이스는 OPMODE_NORMAL 로 부팅한다.	5.1
19	CMD_SET_MODE 는 모든 모드에서 처리되며, 구현되지 않은 모드 값은 ERR_BAD_ARGS 로 거절된다.	5.1
20	OPMODE_NORMAL 에서 CMD_PING, CMD_INFO, CMD_GET_VERSION, CMD_SET_MODE, CMD_DBG_ONLINE, CMD_RESET 에 응답하고 그 밖의 모든 것은 조용히 폐기한다.	5.2
21	기본값이 아닌 모든 모드는 수락된 프레임 없이 ONLINE_TIMEOUT_MS 가 지나면 무조건 OPMODE_NORMAL 로 되돌아간다.	5.3

명령

항목	요구사항	참조
22	§ 6.1 의 모든 명령이 구현되어 있고 모든 모드에서 응답된다.	6.1
23	CMD_GET_VERSION 이 VER_SEL_FW 와 VER_SEL_CMD_SET 을 모두 구현하고, 주어진 선택자를 되돌려주며, 그 밖의 선택자에는 ERR_BAD_ARGS 로 답하고, VER_SEL_CMD_SET 아래에서 § 9.3 의 버전을 보고한다.	6.1, 9.3
24	CMD_RESET 은 리셋하기 전에 응답한다.	6.2
25	OPMODE_NORMAL 밖에서 디바이스가 처리하지 않는 모든 오피코드는 [ERR][ERR_BAD_ARGS] 로, 또는 인식하지 못한 오피코드와 잘못된 인자를 구별할 수 있는 디바이스라면 [ERR][ERR_UNKNOWN] 으로 답한다.	9.1
26	0xC0 이상에는 어떤 명령도 정의되어 있지 않다.	9.2
27	릴리스 안의 명령 집합 버전 상수의 모든 사본이 같은 값을 지닌다.	9.3

스트리밍 — § 7.1 과 § 7.2 를 모두 구현하지 않은 디바이스에는 해당 없음.

항목	요구사항	참조
28	로그 프레임과 텔레메트리 프레임은 SEQ = 0 으로 나간다.	7
29	비요청 프레임의 페이로드는 상태 바이트로 시작하고, 프레임 자신의 내용은 DATA[0] 에서 시작한다.	7, 4.2
30	CMD_LOG_START 는 로깅이 무장되기 전에 확인 응답되며, 그 확인 응답이 어떤 로그 내용보다 앞서 큐에 들어간다.	7.1
31	허용 범위를 벗어난 버스트 주기나 지속 시간은 클램프되지 않고 ERR_BAD_ARGS 로 거절되며, BURST_DURATION_MS_INFINITE 는 받아들여진다.	7.2
32	버스트 프레임은 세워진 마스크 비트마다 u16 하나씩을 비트 오름차순으로 싣고, 연결되지 않았거나 할당되지 않은 비트는 오류 대신 0 을 보고한다.	7.2

항목	요구사항	참조
33	정지 또는 종료 명령은 먹등이고, 호스트가 시작한 경우 요청의 SEQ 를 되돌려주며, 디바이스가 스스로 세션을 끝낼 때에도 SEQ = 0 으로 비요청 전송된다.	6.4, 7.2
34	스트리밍 세션이 활성인 동안에도 명령 디스패치가 계속 동작한다.	7.2

변수와 파라미터 — 두 계열을 모두 구현하지 않은 디바이스에는 해당 없음.

항목	요구사항	참조
35	범위를 벗어난 식별자는 ERR_BAD_ARGS 로 거절된다.	8
36	읽기 전용 슬롯에 대한 쓰기는 ERR_BAD_ARGS 로 거절되며 아무것도 저장하지 않는다.	8
37	디바이스는 쓰기 가능한 각 슬롯에 대해 자신이 받아들이는 범위를 문서화하며, 받아들이지 않는 값은 ERR_BAD_ARGS 로 거절한다 — 클램프하지 않고, 마스킹하지 않으며, OK 로 답하고 버리지 않는다.	8
38	같은 슬롯에 대한 성공한 SET 뒤의 GET 은 설정된 값을 돌려준다.	8
39	슬롯 번호 배정은 덧붙이기 전용이며, 번호를 다시 매기는 경우 MAJOR 명령 집합 버전 변경이 동반된다.	8.1, 9.3
40	한 필드에 둘 이상의 오피코드로 닿을 수 있는 곳에서, 그 오피코드는 쓰기 가능 여부와 받아들이는 값에 대해 일치한다.	8.2

적합한 **호스트** 는 여기에 더해 다음 항목에도 예라고 답한다.

항목	요구사항	참조
41	응답을 SEQ 로 맞추며, 요청 순서를 가정하지 않는다.	3.5
42	자신의 요청과 그에 대한 응답 사이를 포함해 언제든 비요청 프레임이 도착하는 것을 견딘다.	2, 7
43	인식하지 못한 상태 바이트를 프레임 동기를 잃지 않고 견딘다.	4.2
44	비요청 프레임의 첫 페이로드 바이트를 상태 바이트로 읽고, 프레임의 내용은 DATA[0] 에서 취한다.	7
45	손상되어 도착한 프레임에 대해 두 가지 답 — 디바이스의 침묵, 그리고 브리지나 부트로더의 [ERR][ERR_BAD_CRC] — 을 모두 견딘다.	3.7
46	디바이스가 문서화한 최대값보다 큰 프레임을 보내지 않으며, 그 최대값을 실험으로 확립하지 않는다.	3.8
47	어떤 식별자 맵을 쓰기 전에 디바이스의 식별 정보와 명령 집합 버전을 읽고, 그 제품 자신의 맵을 사용한다.	8.1

Celfras Standard Protocol — Part II: 제품

Part I 이 표준이다. 이 부는 Part I 이 각 제품에 남겨 둔 여지를 그 제품이 어떻게 채웠는지에 대한 기록이다. § 8 뒤에 있는 식별자 맵, § 3.8 과 § 5.3 이 제품이 정한다고 선언한 상수의 값, 제품이 어떤 선택적 명령 그룹을 구현하는지, 그리고 제품의 동작이 Part I 이 요구하는 바에서 어디서 벗어나는지.

여기의 어떤 내용도 Part I 의 요구를 완화하지 않으며, 여기의 어떤 것도 새 구현에 요구되지 않는다. 부록은 한 제품에 대한 한 시점의 사실을 진술한다. 그런 사실이 모든 구현자에게 필요한 것으로 드러나면, 그것은 Part I 에 속하며 여기서 인용되는 것이 아니라 그리로 옮겨진다.

이 부가 실는 것과 실지 않는 것. 이 부는 구조를 실는다. 식별자, 이름, 단위, 접근 권한, 그리고 각 슬롯이 무엇을 위한 것인지. 제품이 거기에 써 넣는 숫자는 실지 않는다. 표준은 식별자 14 가 무엇을 뜻하는지를 정의한다. 한 제품이 거기에 무엇을 저장하는지는 그 제품의 사정이며, 그것은 이 문서의 내부용 판과 그 제품 자신의 문서에 실린다. 그 판이 각 슬롯의 내장값을 담은 열을 덧붙이는 자리에, 이 판은 그저 그 열을 가지고 있지 않으며, 이 본문 어디에도 임계값은 나타나지 않는다.

이 규칙은 식별자 맵의 내용에 관한 것이다. 모든 숫자를 금지하는 규칙이 아니다. Part I 이, 호스트가 애초에 대화를 할 수 있도록 디바이스가 자신의 상수를 공표할 것을 요구하는 곳 — § 3.8 의 최대 인코딩 프레임, § 5.3 의 링크 활성화 간격 — 에서는 그 값을 여기에 적는다. 호스트가 그것 없이는 링크를 쓸 수 없기 때문이다. 이 부에 있는 맨숫자는 맵에서 나온 임계값이 아니라 그런 링크 상수 중 하나로 읽으라.

이 부가 어떤 슬롯을 논할 때에는 그 슬롯의 이름을 부르고 번호는 그 위의 표에 맡긴다. 표는 제품 자체로부터 생성된다. 문장 안에 옮겨 적은 id 는 같은 사실의 두 번째 사본이며, 이 부는 이미 번호 재배정을 한 번 겪은 바 있다(A.2).

A. CFS-ECIG-SUITE

Part I 의 참조 구현이며, 이 문서에 생성되는 모든 식별자 표가 추출되어 나오는 제품이다.

A.1 이 제품이 구현하는 것

제품 상수. Part I 은 이 중 둘을 Part II 로 미룬다.

- 받아들이는 가장 큰 인코딩된 프레임(구분자 제외, § 3.8)은 출하되는 CWM2032 빌드에서 **32 바이트** 이다. 누적 버퍼는 같은 펌웨어 안에서도 칩마다 크기가 정해지므로 다른 칩의 빌드는 다른 값을 가진다. 호스트는 소스 트리기가 아니라 보드와 대화하며, 이것이 그 보드의 값이다.
- 링크 활성화 간격 `ONLINE_TIMEOUT_MS` (§ 5.3)는 **5000 ms** 이다.

선택적 그룹. 이 제품은 텍스트 로깅 (§ 7.1), 이진 텔레메트리 (§ 7.2), 튜닝 (§ 6.4), 그리고 변수 계열과 파라미터 계열을 세 폭 모두 (§ 8) 구현한다. 레지스터 접근 (§ 6.3)과 디스플레이 (§ 6.5)는 구현하지 **않는다**. 그 오피코드들은 헤더에 선언되어 있지만 디스패치하는 것이 없으므로, 디바이스가 처리하지 않는 다른 모든 오피코드와 마찬가지로 § 9.1 에 따라 답한다.

§ 6.2 의 이탈은 이 제품의 것이다. `CMD_ACTIVATE`, `CMD_DEACTIVATE`, `CMD_START_HEATING`, `CMD_STOP_HEATING` 이 Part I § 6.2 가 말하는 그 스텝들이다. 무슨 일이 일어나든 일어나지 않든 [OK] 로 답한다. 그 네 오피코드에서 온 [OK] 는 디바이스가 실제로 무언가를 했다는 증거가 아니다.

여기서는 `CMD_SET_MODE` 가 로그 파라미터도 함께 움직인다. `OPMODE_NORMAL` 로 진입하면 A.3 의 `LOG_ENABLE` 파라미터 슬롯이 꺼지고 `OPMODE_DEBUG` 로 진입하면 켜지므로, `OPMODE_DEBUG` 진입 후 첫 로그 줄은 `CMD_LOG_START` 없이 도착한다. 이는 2026-07-10 에 내린 의도적 선택이며, 부록 B 가 대조하는 유일한 동작이다.

A.2 변수 맵

8비트 공간:

ID	Name	Unit
0	EXT_CTRL_STATUS	EXT_CTRL_* bits
1	EXT_HEATING_PWM_DUTY	%
2	OP_MODE	OPMODE_*
3	STATE	STATE_E
4	OIL_MODE	heating profile idx
5	COIL_SEL	H_COIL*_MASK
6	EVENT_FLAG	EVENT_MASK_*
7	EVENT_TOGGLE_FLAG	EVENT_MASK_*
8	BATTERY_GUAGE	%
9	OIL_GUAGE	%
10	EVENT_EMUL_MASK	EVENT_MASK_*
11	EVENT_EMUL_ENABLE	0/1

16비트 공간:

ID	Name	Unit
0	EXT_HEATING_POWER	mW (CP) / mV target_vatrms (CV)
1	PROT_TYPE	PROTECT_MASK_*
2	CHARGING_TIME	min
3	VAT	mV
4	IAT0	mA
5	PAT0	mW
6	RAT0	mohm
7	DUTY0	%

32비트 공간:

ID	Name	Unit
0	STATE_TIME	ms
1	DISP_TIME	ms
2	ONLINE_ELAPSED	ms
3	HEATING_TIME	ms
4	LED_DAISY_STARTED	bursts
5	LED_DAISY_DONE	bursts

이 맵의 대부분은 § 8 이 살아 있는 펌웨어 상태에 대한 읽기 전용 시야라고 서술한 그것이다. 상태 기계의 현재 상태와 그 타이머들, 디바이스가 취하고 있는 측정값들, 계산하고 있는 게이지들. 그중 어느 것에 대한 SET 도

ERR_BAD_ARGS 로 답한다.

다섯 개의 슬롯이 쓰기 가능하며, 그 하나하나가 펌웨어라면 자기 하드웨어에서 읽었을 무언가를 대신하여 호스트가 공급하는 **입력** 이다 — 디바이스 제어 상태의 두 번째 사본이 결코 아니다. 그렇게 하면 양산 제어 코드가 호스트 전용 분기를 늘리는 대신 그대로 실행된다.

- EXT_CTRL_STATUS, EXT_HEATING_PWM_DUTY, EXT_HEATING_POWER 는 외부 제어 인터페이스이다. 그 비트들이 무엇을 뜻하는지, 각 슬롯이 무엇을 받아들이는지, 그리고 그것을 구동하는 동안 호스트가 무엇을 해야 하는지는 여기가 아니라 제품 자신의 문서에 있다.
- EVENT_EMUL_MASK 와 EVENT_EMUL_ENABLE 은 호스트가 디바이스 자신의 이벤트 입력을 대신하게 해 준다. 둘 다 자기 슬롯의 폭 전체를 받아들인다.

호스트가 어떤 값을 신뢰하기 전에 알아야 할 사실이 세 가지 더 있다.

- **16비트 맵의 측정 부분은 가열 코일이 하나임을 전제한다.** 코일별 슬롯의 이름이 코일 0 을 가리키는 것은 이 보드에 그 하나뿐이기 때문이다. 코일이 둘인 보드는 이 슬롯들의 번호를 다시 매기는 대신 코일 1 의 슬롯을 그 뒤에 덧붙인다.
- **빌드에 따라 어떤 슬롯은 측정값이 아니라 0 을 읽을 수 있다.** RAT0 은 정전압 빌드에서 읽을 전류 감지 채널이 없고, DISP_TIME 은 데이지체인 디스플레이 빌드에서 상태별 타임라인이 없다. 거기서의 0 은 연결되지 않은 텔레메트리 비트(§ 7.2) 에서와 똑같이 *이 빌드에서는 연결되지 않았다* 는 뜻이며, 0 이라는 측정값이 아니다.
- **변수 번호가 지금까지 만든 보드들에서 우연히 같을 뿐이며, 그것은 보장이 아니라 우연이다.** § 8.1 은 파라미터 맵에 적용되는 것과 똑같이 이 맵에도 적용된다. 그 일치는 역사의 우연이고, 다음 보드에 대해 아무것도 약속하지 않으며, 어떤 것도 안전하게 만들지 않는다. 호스트는 어느 맵을 쓰든 그 전에 자신이 어떤 제품과 대화하는지를 여전히 확립한다.

이 맵은 2026-07-29 에 한 번 번호가 다시 매겨졌다. 외부 제어 블록을 뒤에 덧붙이는 대신 8비트 공간과 16비트 공간의 앞에 끼워 넣었을 때이다. 그 시점은 부록 E 의 명령 집합 번호 부여보다 앞서므로 그것을 표시하는 버전 차이가 없다. § 8.1 의 덧붙이기 전용 규칙은 그 이후로 지켜지고 있으며, 옛 사본의 맵은 엉뚱한 슬롯을 가리킨다.

A.3 파라미터 맵

8비트 공간:

ID	Name	Unit	Access
0	LOG_ENABLE	0/1	rw

16비트 공간:

ID	Name	Unit	Access
0	VDD	mV	ro
1	VAT0	mV	ro
2	IAT0	mA	ro
3	LED_BREATH_PERIOD_MS	ms	rw
4	UVLO_HEAT_INIT_TH	mV	rw

ID	Name	Unit	Access
5	UVLO_HEAT_TH	mV	rw
6	LONG_PUFF_TH	ms	rw
7	CHG_OVP_TH	mV	rw
8	CHG_TIMEOUT_TH	min	rw
9	SHORT_COIL1_TH	mV	rw
10	SHORT_COIL2_TH	mV	ro
11	SHORT_COIL_RST_TH	mV	rw
12	OPEN_COIL1_TH	mV	rw
13	OPEN_COIL2_TH	mV	ro
14	DRY_PUFF_ABS_TEMP_TH	degC	rw
15	DRY_PUFF_REL_SLOPE_TH	degC/sec	rw

32비트 공간:

ID	Name	Unit	Access
0	HEATING_TIMEOUT_TH	ms	rw

이 번호들은 이 제품의 것이며 이식 가능하지 않다. 이것이 Part II 에서 가장 결과가 큰 사실이며, 그 뒤에 있는 규칙을 § 8.1 이 말한다. 모든 보드가 자기 파라미터 열거를 독립적으로 번호 매기므로, 같은 id 가 다른 제품에서는 다른 필드를 가리킨다 — 그리고 그것을 검출하는 것이 아무것도 없다. 엉뚱한 보드를 향한 호스트는 엉뚱한 측정값을 읽고, 더 나쁘게는 엉뚱한 임계값을 쓴다. 펌웨어는 그 쓰기를 수행하고 **OK** 로 답한다. 어떤 계층에도 오류가 없고, 어긋나는 체크섬도 없으며, 나중에 찾아볼 로그도 없다. 유일한 방어는 § 8.1 이 요구하는 것이다 — 식별 정보와 명령 집합 버전을 먼저 읽고, 그 제품 자신의 맵을 쓴다.

쓰기 가능한 슬롯 대부분은 튜닝 빌드에서만 열린다. 보호 임계값들 — 16비트와 32비트 맵에서

LED_BREATH_PERIOD_MS 를 제외한 모든 rw 슬롯 — 은 디바이스의 파라미터 블록에 쓰는데, 그 블록은 릴리스 빌드에서 const 이다. 거기서는 그 하나하나가 저장하는 대신 ERR_BAD_ARGS 로 답한다. 그것은 출하되는 제품의 의도된 동작이며, 결함도 퇴행도 아니다. 릴리스 빌드란 임계값이 제품이 교정받은 그 값인 빌드이다. 위의 표들은 각 슬롯의 접근 권한을 튜닝 빌드 기준으로 준다. 맵을 문서화할 값어치가 있는 쪽이 그쪽이기 때문이다. 쓰기 가능한 두 슬롯은 파라미터 블록에 전혀 손대지 않으므로 영향을 받지 않는다. LOG_ENABLE 과 LED_BREATH_PERIOD_MS 는 서비스 호출을 거치며 모든 빌드에서 쓰기 가능한 상태로 남는다.

두 슬롯은 이 보드에 없는 두 번째 가열 코일을 위해 예약되어 있다. SHORT_COIL2_TH 와 OPEN_COIL2_TH 는 번호가 이미 두 코일 빌드와 맞도록 선언되어 있다. 이들은 모든 빌드에서 0 을 읽고 모든 쓰기를 거절한다 — 이는 위의 튜닝 빌드 효과가 아니며, 튜닝 빌드를 선택해도 열리지 않는다.

개별 슬롯에 관한 더 작은 사실 네 가지:

- SHORT_COIL1_TH 와 OPEN_COIL1_TH 는 한 쌍이며, 펌웨어는 이들을 각각 고정된 경계와 비교하지 않고 서로에 대해 비교한다. 둘 다 같은 측정 강하에 대해 시험되며, 한 임계값 아래는 개방 코일 고장, 다른 임계값 위는 단락 코일 고장이다. 따라서 둘의 순서는 호스트가 무엇을 쓸 수 있는지에 대한 제약이다 — 단

락 코일 임계값은 개방 코일 임계값보다 위에 있어야 한다. 뒤집히면 그 사이의 정상 구간이 두 고장이 동시에 걸리는 구간이 되므로, 뒤집는 것은 코일 검출을 조이는 것이 아니라 무력화하는 것이다.

- CHG_OVP_TH 와 SHORT_COIL_RST_TH 는 읽고 쓸 수 있지만 현재 이들과 비교하는 것이 아무것도 없다. 하나는 비교가 비활성화된 분압 값이고, 다른 하나는 다른 수단으로 검출되는 보호에 속한다. 번호 안정성을 위해 남겨 두었으며, 오늘 이들에 쓰는 것은 아무것도 바꾸지 않는다.
- LED_BREATH_PERIOD_MS 는 **저장하지 않는 0 을 받아들인다**. 0 은 디스플레이 램프에서 0 으로 나누기가 되므로 설정 함수가 일찍 반환하지만 명령은 여전히 [OK] 로 답하며, 그 뒤의 GET 은 이전 값을 돌려준다. 이는 § 8 의 되읽기 규칙에서 벗어나는 것이며, 덮지 않고 여기에 기록한다. 받아들이는 범위는 1 이상이며, 새 구현은 이를 따라 하지 말고 그 0 에 ERR_BAD_ARGS 로 답하여야 한다.
- LOG_ENABLE 은 **주어진 것과 다른 것을 저장한다**. 설정 함수가 0 이 아닌 모든 값을 1 로 줄이고, 명령은 그와 무관하게 [OK] 로 답한다. 따라서 0 이 아닌 어떤 값의 SET 도 OK 로 답해지고 그 뒤의 GET 은 1 을 돌려준다. 이것이 § 8 이 금지하는 마스킹이며, 바로 위 슬롯과 같은 결함을 반대쪽에서 본 것이다 — 응답을 구성하는 계층에 거부를 보고할 방법이 없는 설정 함수, 그래서 OK 라고 말하는 응답. 받아들이는 범위는 0 과 1 이며, 새 구현은 이를 따라 하지 말고 그 밖의 값에 ERR_BAD_ARGS 로 답하여야 한다. 이 둘은 두 사례를 가진 하나의 결함이며, 이 제품 자신의 결함 목록에 무관한 두 개의 별난 점이 아니라 하나의 항목으로 기록되어 있다.

이 둘을 빼면, 쓰기 가능한 파라미터 슬롯은 자기 슬롯 폭 전체를 받아들인다. 펌웨어는 값이 아니라 식별자를 범위 검사한다. 디바이스가 제대로 동작하려면 어떤 값이 온당해야 하는 곳에서 그 제약은 제품의 것이며, 제품 자신의 문서에 있다.

A.4 텔레메트리 필드

Part I § 7.2 는 어떤 비트가 어떤 양인지를 고정하고 두 가지를 제품에 맡긴다. 각 필드 뒤의 스케일, 그리고 어떤 비트가 실제 측정에 연결되어 있는지. 이 제품에서 모든 필드는 u16 이며, 스케일은 다음과 같다.

- LOG_FIELD_VDD, LOG_FIELD_VAT — 밀리볼트.
- LOG_FIELD_IAT — 밀리암페어.
- LOG_FIELD_PWR — 밀리와트. 직접 측정하지 않고 측정된 부하 전압과 전류, 그리고 구동의 듀티 사이클로부터 산출한다.
- LOG_FIELD_DUTY — 퍼센트.
- LOG_FIELD_RAT — 밀리움. 측정이 아니라 디바이스가 계산한다.
- LOG_FIELD_PROT — 디바이스 보호 상태의 비트필드이며, 스케일이 있는 양이 아니다.

실제 측정에 연결된 것: VDD, VAT, IAT, PWR, DUTY, 그리고 드라이 퍼프를 지원하는 정전력 빌드에서의 RAT. PROT 은 연결되어 있지 않으며, RAT 은 그것을 계산할 전류 감지 채널이 없는 정전압 빌드에서 0 을 읽는다. 연결되지 않은 비트도 여전히 받아들여지고 여전히 프레임 안에서 자기 u16 자리를 차지하며 0 을 보고한다. § 7.2 가 요구하는 그대로이다.

A.5 튜닝 식별자

Part I § 6.4 는 세션의 모양을 고정하고 식별자는 제품에 맡긴다. 이 제품은 네 개를 정의하며, CMD_TUNING_START 는 실행 중인 빌드가 지원하지 않는 식별자를 ERR_BAD_ARGS 로 거절한다.

- TUNING_ID_CV_VATRMS (0) — 정전압 구동 목표를 훑는다. 정전압 빌드 전용.
- TUNING_ID_BATTERY_GAUGE (1) — 배터리 게이지의 전압 구간을 훑는다. 방전 위상과 충전 위상에 대해 정의되어 있으나, **출하 펌웨어는 충전 위상만 실행한다**. 세션이 어느 위상을 취하는지는 충전기가 실제로

붙어 있는지로부터 디바이스가 결정한다. 붙어 있지 않으면 세션은 방전 스위치가 될 것이므로, CMD_TUNING_START 는 그 식별자를 아예 ERR_NOT_READY 로 거절한다. 이 거절은 의도적이다. 방전 스위치의 프레임은 셀 자신의 전압이 아니라 부하가 걸린 전압을 보고하게 되므로, 그 스위치는 교정처럼 보이지만 교정이 아닌 숫자를 돌려주게 된다. 거절이 정직한 답이다. 원인은 이 제품 측정 경로의 퇴행이며 그 제품 자신의 결함 목록에 기록되어 있다. 이 거절은 아래의 준비 상태 거절이 아니며 — 호스트가 미리 준비해 둔 어떤 것에도 의존하지 않는다 — 오류 코드만을 공유한다.

- TUNING_ID_PROTECT_THRESH (2) — 단락 코일과 개방 코일의 저항 임계값을 훑는다.
- TUNING_ID_DRYPUFF (3) — 드라이 퍼프의 온도 임계값과 기울기 임계값을 훑는다. 드라이 퍼프를 지원하는 정전력 빌드 전용.

세션은 A.2 와 같은 외부 제어 입력 경로를 통해 실제 양산 제어 알고리즘을 실행하므로, 호스트는 세션을 시작하기 전에 자극을 미리 준비해 둔다. 그 준비는 그 인터페이스와 함께 제품 자신의 문서에 속한다.

CMD_TUNING_START 는 준비된 자극으로는 어떤 측정도 나올 수 없는 세션을, 끝까지 실행하고 아무것도 보고하지 않는 대신 ERR_NOT_READY 로 거절한다.

한 식별자는 요청된 보고 주기도 제약한다. 배터리 게이지 세션은 자신의 고정된 표본 주기의 한 사이클마다 한 번 보고하므로, 주기가 제어할 것이 남아 있지 않다. CMD_TUNING_START 는 다른 값을 받아들여 다른 속도로 보고하는 대신 ERR_BAD_ARGS 로 거절한다. 한 주기를 요청하고 조용히 다른 주기를 받은 호스트는 타임스탬프 사이의 모든 간격을 잘못 읽게 되기 때문이다. 그 주기 자체는 이 제품의 상수이며 제품 자신의 문서에 적혀 있다. 그 밖의 어떤 식별자도 § 6.4 가 요구하는 것 이상으로 주기를 제약하지 않는다.

인덱스 조합의 의미. § 6.4 는 CMD_SET_TUNING_PAR/CMD_GET_TUNING_PAR 인덱스의 의미를 제품에 맡긴다. 여기서는 tuning_id 가 교정 표를 선택하고 나머지 인덱스가 그 안의 칸을 지정한다. 각 식별자는 자기 표가 차원을 가진 인덱스만 사용하며, 사용하는 모든 인덱스는 실제 차원에 대해 범위 검사를 받고, 위반은 § 6.4 가 요구하는 대로 ERR_BAD_ARGS 로 답한다. 두 명령 모두 세션을 필요로 하지 않는다.

- TUNING_ID_CV_VATRMS 만이 조합 전체를 사용한다. coil_idx 는 가열 코일, heating_mode 는 구동 모드, heater_type 은 장착된 히터, step 은 목표 전압 단계이다.
- TUNING_ID_BATTERY_GAUGE 는 coil_idx 를 **충방전 방향** 으로 — 물리적 코일이 아니라 방전인지 충전인지로 — 쓰고, step 을 게이지 구간으로 쓴다.
- TUNING_ID_PROTECT_THRESH 는 coil_idx 를 가열 코일로 쓰고, step 으로 그 쌍 중 어느 쪽을 — 단락 코일 임계값인지 개방 코일 임계값인지를 — 지정한다.
- TUNING_ID_DRYPUFF 는 step 으로 절대 온도 임계값과 기울기 임계값 중 하나를 선택한다. 이들은 코일 별이 아니지만 coil_idx 는 여전히 코일 수에 대해 검사되므로, 존재하지 않는 코일을 지목한 호스트는 그냥 처리되는 대신 그 사실을 듣는다.

식별자가 사용하지 않는 인덱스는 0 이어야 한다고 요구되지 않고 무시되며, 범위 검사도 받지 않는다. 호스트는 그 자리에 0 을 보낼 것을 권장한다. 나중에 그 차원에 의미를 부여하는 제품은 그것을 검사하기 시작할 것이기 때문이다.

세션이 어떤 보호를 중단시키는가 — § 6.4 가 요구하는 공개 사항이다. 긴 흡입 보호와 가열 타임아웃 보호는 호스트가 가열 트리거를 구동하고 있는 동안 언제나 중단되며, 여기에는 튜닝 세션뿐 아니라 평범한 외부 제어도 포함된다. 충전 타임아웃 보호는 배터리 게이지 세션 동안 중단된다. 셀의 전 범위에 걸쳐 충전하는 것이 곧 그 측정이기 때문이다. 그리고 코일 저항 검사는 보호 임계값 세션 동안 중단된다. 그 스위치야말로 그 검사가 거절하려고 존재하는 값에 도달해야 하는 스위치이기 때문이다. 저전압 차단과 충전 과전압 보호는 그 동안에도 살아 있다. 지나치게 길어진 외부 구동 가열 요청을 끝내는 데에 긴 흡입 보호나 가열 타임아웃 보호를 의지하고 있던 호스트에게는 그 안전망이 없다.

이 중단 중 하나는 읽히는 만큼 멀리 미치지 못한다. 코일 저항 검사의 중단은 디바이스가 가열 중에 수행하는 검사를 중단시킬 뿐, 가열 시작 시점에 수행하는 검사는 중단시키지 않는다. 후자는 핀 판독으로부터 곧바로 개방 코일 고장을 일으키며, 그 경로에는 중단이 닿는 것이 아무것도 없다. 따라서 그 검사가 거절할 부하에 대한 보호 임계값 세션은 여전히 가열 시작에서 멈춰 세워질 수 있다 — 바로 그 중단이 허용하려고 존재하는 경우이다. 이는 이 제품 자신의 결함 목록에 기록되어 있으며, 어떤 중단을 공개하는 제품은 그 공개가 포괄하는 모든 검사에 그 중단이 미치게 할 것을 권장한다.

이 제품의 프로토콜 표면에 있는 불일치 하나는 호스트에게 보이는 것이므로 적어 둘 값어치가 있다. 정전압 튜닝 빌드에서 두 개의 드라이 퍼프 임계값 슬롯은 `CMD_PAR16_SET` 을 통해서 쓰기 가능하며 `OK` 로 답하고, `CMD_SET_TUNING_PAR` 을 통해서 그 빌드에서 그것을 읽는 것이 아무것도 없다는 올바른 근거로 `ERR_BAD_ARGS` 로 거절된다. 두 오피코드, 한 필드, 반대되는 답. 이는 한 필드에 닿는 두 오피코드가 쓰기 가능 여부에 대해 일치할 것을 요구하는 § 8.2 로부터의 이탈이다. 필드는 어느 쪽이든 존재하므로 이는 프로토콜 표면만의 불일치이며, 기록되었을 뿐 아직 해소되지 않았다.

B. CVS-BP2601

같은 표준 위의 두 번째 제품. 그 식별자 맵은 그 제품 자신의 것이며 그 제품과 함께 보관된다. § 8.1 이 그 맵들에 온전히 적용되며, 부록 A 를 그 맵에 대한 서술로 읽어서는 안 된다.

기록된 차이는 정확히 하나이며, A.1 이 미리 마련해 둔 그것이다. **CVS-BP2601 의 `CMD_SET_MODE` 는 로그 활성화 파라미터를 건드리지 않는다.** 거기서는 `OPMODE_DEBUG` 로 진입해도 명시적인 `CMD_LOG_START` 가 있기 전까지 로깅이 꺼져 있는 반면, CFS-ECIG-SUITE 는 모드와 함께 그것을 켜다 켜다 한다. 두 동작 모두 적합하다 — § 5 는 모드가 무엇을 위한 것인지를 정의할 뿐 파라미터 슬롯을 모드에 묶지 않는다 — 따라서 어느 제품에서든 로그 출력을 원하는 호스트는 자신이 선택한 모드로부터 그것을 추론하지 말고 명시적으로 요청한다. CFS-ECIG-SUITE 의 동작은 그 프로젝트에서 2026-07-10 에 내린 의도적 선택이며 이쪽으로 이식되지 않았다.

C. 브리지 계층

브리지는 § 1.3 의 중계기이며, § 9.2 가 이미 디바이스 쪽에서 본 규칙을 말한다. `0xc0` 이상은 이 계층의 것이고, 제품은 거기에 명령을 정의해서는 안 되며, 브리지는 오피코드 바이트만으로 경로를 정한다. 이 부록은 그 규칙이 대역을 예약해 둔 대상, 즉 그 표이다.

ID	Name
0xc0	CMD_B_PING
0xc1	CMD_B_INFO
0xc2	CMD_B_GET_CAPS
0xc3	CMD_B_GET_STATS
0xc4	CMD_B_GET_SNAPSHOT
0xc5	CMD_SET_SWD_SPEED
0xc6	CMD_SET_TARGET
0xc7	CMD_B_GET_VERSION

ID	Name
0xC8	CMD_MODE_DEBUG
0xC9	CMD_MODE_DOWNLOAD
0xCA	CMD_MODE_RAW
0xCB	CMD_MODE_BRIDGE_RESET
0xCC	CMD_TGT_NRST
0xCD	CMD_TGT_POWER
0xCE	CMD_BRIDGE_CONF
0xD0	CMD_SWD_CONNECT
0xD1	CMD_SWD_DISCONNECT
0xD2	CMD_SWD_LINE_RESET
0xD3	CMD_SWD_JTAG2SWD
0xD4	CMD_SWD_READ_IDCODE
0xD7	CMD_SWD_CONNECT_UNDER_RESET
0xD8	CMD_DP_READ
0xD9	CMD_DP_WRITE
0xDA	CMD_AP_READ
0xDB	CMD_AP_WRITE
0xE0	CMD_CORE_HALT
0xE1	CMD_CORE_RESUME
0xE2	CMD_CORE_STATUS
0xE3	CMD_CORE_SYSRESET
0xE4	CMD_CORE_RUN_DETACH
0xE8	CMD_UPD_BEGIN
0xE9	CMD_UPD_DATA
0xEA	CMD_UPD_END
0xEB	CMD_UPD_STATUS
0xEC	CMD_UPD_ABORT
0xED	CMD_UPD_APPLY
0xF0	CMD_FLASH_ERASE_SECTOR
0xF1	CMD_FLASH_PROG_PAGE
0xF2	CMD_FLASH_PROG_PAGE_VERIFY
0xF3	CMD_FLASH_READ
0xF4	CMD_FLASH_MASS_ERASE
0xF5	CMD_FLASH_READ_EX

ID	Name
0xF8	CMD_OB_READ
0xF9	CMD_OB_WRITE
0xFA	CMD_OB_LAUNCH
0xFB	CMD_OB_WRITE_AND_LAUNCH

이 표를 한 보드의 표가 아니라 한 계층의 표로 읽으라. 두 종류의 브리지 보드가 현업에 쓰이고 있다. 둘은 성능에서 다를 뿐 무엇에 응답하는지에서는 다르지 않으며, 둘 다 이 표를 선언한다. 그것이 § 9.2의 예약을 보드별 관행이 아니라 예약으로 만들어 준다.

이 표를 쓰기 전에 호스트가 알아야 할 것 네 가지:

- **일부 항목은 처리기 없이 예약만 되어 있으며**, 표는 어느 것이 그런지 표시하지 않는다. 어떤 오피코드에 처리기가 있는지는 대역의 표가 아니라 응답하는 이미지의 속성이다. 그것은 펌웨어 버전에 따라 달라지며, 부트로더는 애플리케이션보다 훨씬 적은 오피코드에 응답한다. 예약된 오피코드는 구현되지 않은 다른 오피코드와 마찬가지로 ERR_BAD_ARGS로 답해지므로 (§ 9.1), 호스트는 버전을 먼저 읽는 대신 그것을 탐침한다.
- **CMD_UPD_*(0xE8-0xED)는 브리지 자신을 갱신하며, 오직 브리지의 부트로더만 응답한다.** 이들은 브리지 뒤에 있는 디바이스를 프로그래밍하는 CMD_FLASH_* 블록이 아니다. 둘을 혼동하는 것은 프로토콜 수준의 방어가 없는 실수이다. 두 블록 모두 같은 이 대역에 있고 둘 다 그럴듯하게 응답한다. 그 모양은 다음과 같다. CMD_UPD_BEGIN은 고정 512 바이트 이미지 헤더를 받고 [OK][chunk_max u16][capacity u32]로 답한다. CMD_UPD_DATA는 [off u32][data]를 받되 데이터는 최대 256 바이트이고 방금 돌려받은 chunk_max를 결코 넘지 않으며, [OK][next_off u32]로 답한다. CMD_UPD_STATUS는 페이로드를 받지 않고 [OK] 뒤에 고정 18 바이트 상태 블록을 붙여 답하며, 그 필드 배치는 브리지 자신의 것이고 브리지와 함께 문서화된다. CMD_UPD_END, CMD_UPD_ABORT, CMD_UPD_APPLY는 페이로드를 받지 않고 [OK]로 답하며, 보드는 CMD_UPD_APPLY에 응답한 뒤에 이미지를 복사하고 스스로 리셋한다. 이 크기들은 이 블록의 선 위 상수이며 모든 브리지 보드에서 동일하다.
- **CMD_BRIDGE_CONF는 여러 설정을 담당하는 하나의 오피코드이며**, 파라미터 바이트로 구분된다. 이 대역에는 남은 번호가 적고, 토큰 하나가 번호 하나를 소비해서는 안 되기 때문이다. 알 수 없는 파라미터는 ERR_BAD_ARGS로 답해지며, 호스트는 그것으로 버전을 읽지 않고도 어떤 설정이 지원되는지 검출한다.
- **CMD_B_PING과 CMD_B_INFO가 존재하는 이유는 디바이스 자신의 CMD_PING과 CMD_INFO가 누가 응답했는지를 말해 줄 수 없기 때문이다.** 두 계층이 그 명령들에 같은 방식으로 응답하므로, 브리지/가 거기 있는지 확인하려는 호스트는 브리지 자신의 대역에서 묻는다.

브리지의 버전 명령은 § 6.1이 경고하는 대로 자신만의 선택자 공간을 지닌다. 같은 선택자 번호가 디바이스의 것과 다른 것을 뜻한다. 펌웨어 버전 외에 이 명령은 실행 중인 이미지의 **역할**에 답한다. 이는 애플리케이션과 부트로더를 구별해 준다 — 펌웨어 버전만으로는 어느 이미지가 그것을 냈는지 알 수 없으므로 호스트가 가장 먼저 물어야 할 것이다 — 그리고 보드의 **하드웨어 식별 정보**에도 답한다. 이것은 어느 이미지에서 물어도 같은 답이 나와야 하는 유일한 선택자이다. 이미지가 바뀌어도 하드웨어는 바뀌지 않기 때문이다.

역할 선택자는 호스트가 명령 집합 선택자에 무엇을 기대해야 하는지를 배우는 자리이기도 하다. 브리지 부트로더는 그 선택자에 자기 자신의 번호로 답하며, 그 번호는 의도적으로 애플리케이션의 번호보다 낮게 고정되어 있다. 그것은 부트로더 자신이 구현하는 몇 안 되는 오피코드를 마지막으로 바꾼 개정을 가리키며, 애플리케이션의 번호가 움직여도 따라 움직이지 않는다. 방금 역할을 읽어 부트로더임을 확인한 호스트는 다음에 읽을 명령 집합

버전이 더 낮으리라고 예상해야 하며, 그 차이를 결함이나 잘못 빌드된 릴리스로 취급하여서는 안 된다. § 9.3의 릴리스 장부 규칙은 한 이미지가 지닌 상수의 사본들에 관한 것이며, 부트로더는 다른 이미지이다.

브리지의 오류 코드는 § 4.4의 확장 공간에 살며, § 4.4가 그 블록들을 가리킨다. 여기에 덧붙일 모양이 하나 있다. 모든 `ERR_UPD_*` 코드는 코드 바이트 뒤에 u32 상세 정보를 싣는다. 실측 현장에서 유용한 질문은 갱신이 실패했는가가 아니라 어느 오프셋에서, 어느 주소에서, 어느 검사에서 실패했는가이기 때문이다.

C.1 CMSIS-DAP 페르소나

`CMD_MODE_BRIDGE_RESET`을 대상 `0x02(BRIDGE_RESET_TO_DAP)`로 받은 브리지는 응답하고, 리셋한 뒤, CMSIS-DAP v1 디버그 장치로 재열거한다: USB HID 전용, VID `0x0A05`, PID `0x050B`, 64 바이트 IN 리포트 하나와 64 바이트 OUT 리포트 하나, CMSIS-DAP을 포함하는 제품 문자열, CDC 페르소나와 같은 `iSerialNumber`. CMSIS-DAP 명령 자체는 ARM의 것이며(CMSIS-DAP 2.1.1 명령 집합, SWD 전용, `0x00-0x13`), 이 표준은 그것을 다시 기술하지 않는다.

벤더 명령 `0x80(DAP_Vendor0)`은 *CFS 터널*이다: 요청 리포트 `[0x80][n][n 바이트]` ($n \leq 62$)는 CDC 페르소나가 쓰는 것과 같은 COBS 바이트 스트림의 다음 n 바이트를 싣고, 응답 리포트 `[0x80][m][m 바이트]`는 응답 스트림의 다음 m 바이트를 돌려준다. $n = 0$ 은 더 많은 응답 바이트를 요청(poll)한다. 프레임링, CRC, 오피코드는 바뀌지 않는다. 이 페르소나에서 브리지는 `CMD_B_PING`, `CMD_B_INFO`, `CMD_B_GET_CAPS`, `CMD_B_GET_VERSION`, `CMD_B_GET_STATS`, `CMD_B_GET_SNAPSHOT`, 그리고 `CMD_MODE_BRIDGE_RESET`(모든 대상)만 응답한다. 그 밖의 모든 오피코드, 그리고 모든 DUT 오피코드는 `ERR_NOT_READY`로 답한다.

페르소나는 RAM에 보관된다: 전원 순환은 언제나 브리지를 CDC 페르소나로 되돌리며, 보드의 KEY 버튼도 마찬가지다.

C.2 브리지 기능 비트 (CMD_B_GET_CAPS)

2.0.0부터 예약만 되어 있던 `CMD_B_GET_CAPS(0xC2)`는 브리지 명령 집합 3.0.0부터 구현된다. 요청은 페이로드가 없고, 응답은 `[OK][caps u32 LE][reserved u32 LE]`이다. 두 번째 워드는 0이며 호스트는 무시한다. 이 명령은 어떤 핀도 건드리지 않으므로 어느 모드에서든, 어느 페르소나에서든 응답하며, 호스트는 다른 무엇보다 먼저 보내도 된다.

`caps`는 *응답하는 펌웨어*가 무엇을 할 수 있는지를 말한다. 어느 보드에서 도는지를 말하는 `CMD_B_GET_VERSION(VER_HW)`와는 다르다. 호스트는 이 비트로 분기하며, 자기 나름의 보드별 표로 분기하지 않는다. 비트는 재할당되지 않으며, 비트 8-31은 미할당이고 0으로 읽힌다.

- `BCAP_TGT_NRST(0x00000001)` — `CMD_TGT_NRST`가 실제 nRESET 선을 구동한다.
- `BCAP_TGT_POWER_OEN(0x00000002)` — `CMD_TGT_POWER`가 존재한다: 브리지가 DUT 전원을 켜고 끌 수 있다(BRD02에서는 핀 자체가 레일이며 쓰기는 `BCONF_TGT_POWER_EN`으로 게이트된다. BRD01에서는 패스 트랜지스터가 USB VBUS를 게이트한다).
- `BCAP_PIN_MAP(0x00000004)` — `BCONF_PIN_MAP`으로 런타임 핀 맵을 고를 수 있다.
- `BCAP_FLASH_READ_EX(0x00000008)` — `CMD_FLASH_READ_EX`가 구현되어 있다.
- `BCAP_DAP_PERSONA(0x00000010)` — `BRIDGE_RESET_TO_DAP`으로 C.1의 CMSIS-DAP HID 페르소나로 재부팅한다.
- `BCAP_BTN_HW_RESET(0x00000020)` — 보드에 하드웨어 리셋 버튼(KEY)이 있다.
- `BCAP_TGT_POWER_5V(0x00000040)` — 스위칭되는 DUT 전원이 5.0 V다. 0이면 3.3 V IO 레일이다.

- `BCAP_GPIO_CTRL(0x00000080)` — 브리지 GPIO 제어 명령을 위해 예약. 그런 명령은 아직 없으므로 모든 보드가 0 으로 답한다.

빌드된 값으로 `BRD02` 는 `0x3F`(5 V 레일만 빠진다. 스위칭 레일이 3.3 V IO 이므로), `BRD01` 은 `0x4B`(`nRESET`, 전원 스위칭, `CMD_FLASH_READ_EX`, 5 V 레일. 런타임 핀 맵, DAP 페르소나, KEY 는 없다)로 답한다.

3.0.0 미만의 앱은 알 수 없는 오피코드로 `ERR_BAD_ARGS` 를, 브리지 부트로더는 미구현 오피코드에 대한 기본 응답인 `ERR_NOT_READY` 를 답한다. 호스트는 어떤 오류 응답이든 `caps = 0` 으로 다룬다. 브리지가 말할 수 없으니 어떤 선택 기능도 있다고 가정하지 않는다.

C.3 플래시 단위는 대상의 것이다 (`CMD_SET_TARGET`)

브리지는 뒤에 있는 디바이스를 그 디바이스 자신의 단위로 프로그램하고 소거하며, 그 단위는 패밀리마다 다르다. `CWM0508`, `CWM0524`, `CWM1016`, `CWM2032` 패밀리는 128 바이트 페이지를 프로그램하고 4 KB 섹터를 소거한다. `CWM30C8` 패밀리(모델 `0x04`, 명령 집합 3.1.0)는 256 바이트 페이지를 프로그램하고 8 KB 섹터를 소거한다. 그러므로 플래시 블록의 세 오피코드는 자신의 크기를 갖지 않는다.

- `CMD_FLASH_ERASE_SECTOR(0xF0)`는 `[addr u32]` 를 받아 `addr` 을 포함하는 섹터를 소거한다. 정렬은 브리지가 내림으로 한다.
- `CMD_FLASH_PROG_PAGE(0xF1)`와 `CMD_FLASH_PROG_PAGE_VERIFY(0xF2)`는 `[addr u32]` 뒤에 선택된 대상의 페이지 정확히 하나를 받는다. 그 외의 바이트 수를 실은 프레임에는 `ERR_BAD_ARGS` 로 답하여야 한다. 검증형의 오류 응답은 `[err_code][mismatch_off u8]` 이며, 페이지 안에서 처음 다른 바이트의 오프셋이다.

호스트는 단위를 자기 표가 아니라 브리지에게서 배운다. 3.1.0 부터 `CMD_SET_TARGET` 응답은 이미 답하던 플래시 베이스와 크기 뒤, 오프셋 20 에 `[page_size u16 LE]` `[sector_size u16 LE]` 를 실는다. 3.1.0 미만의 브리지는 20 바이트를 답하며 단위가 128 B / 4 KB 가 아닌 패밀리를 알지 못하므로, 20 에서 읽기를 멈추는 호스트는 그 브리지에서 읽는 것이 없다. 그리고 브리지가 모르는 모델을 고른 호스트는 페이지를 하나도 보내기 전에 `ERR_BAD_ARGS` 를 받으므로, 256 바이트 페이지가 128 을 프로그램하는 브리지에 닿는 일은 없다.

3.1.0 이전에 이 세 오피코드의 이름은 `CMD_FLASH_ERASE_4K`, `CMD_FLASH_PROG_PAGE_128`, `CMD_FLASH_PROG_PAGE_128_VERIFY` 였다. 번호는 움직이지 않았다. 이름이 크기를 주장하기를 그만두었을 뿐이다.

D. DUT 시험 펌웨어

브리지가 언제나 대화할 적합한 디바이스를 갖도록 네 개의 작은 펌웨어가 존재하며, 브리지가 상대하는 칩마다 하나씩이다. **CWM2032**, **CWM1016**, **CWM0508**, **CWM30C8**. 이들은 제품이 아니라 시험 대상이며 어디에도 출하되지 않는다.

각각은 § 5.2 가 `OPMODE_NORMAL` 에서 응답하도록 요구하는 여섯 개의 오피코드 — `CMD_PING`, `CMD_INFO`, `CMD_GET_VERSION`, `CMD_SET_MODE`, `CMD_DBG_ONLINE`, `CMD_RESET` — 만을 정확히 구현하고 그 밖에는 아무것도 구현하지 않으며, 각각 제품들과 같은 명령 집합 버전을 선언한다. 그 이유는 § 9.3 이 말하는 것이다. 이 버전은 오피코드 표를 가리키는 것이지 그 뒤에 있는 구현의 크기를 가리키지 않는다. 여섯 개의 오피코드를 구현하고 나머지에 § 9.1 에 따라 답하는 디바이스는 전부를 구현한 디바이스만큼 적합하다.

E. 명령 집합 변경 이력

하나의 버전 번호가 두 대역을 모두 포괄한다(§ 9.3). 이 번호는 다음 목록의 모든 곳에 중복되어 있다.

- CFS-ECIG-SUITE 펌웨어와 그 호스트 도구.
- 두 브리지 펌웨어 각각, 그 두 보드의 빌드 메타데이터 파일 각각, 그리고 브리지 호스트 도구.
- 부록 D 의 네 시험 펌웨어 각각.

이들 중 어느 것도 다른 것에 대해 권위를 갖지 않는다. 이들이 서로 어긋나는 릴리스는 잘못 빌드된 것이며, 그것이 § 9.3 이 말하는 릴리스 장부 규칙이다. 목록이 곧 사실이며, 그 옆에 총 개수는 적지 않는다. § 9.3 은 이 사본들을 가정하지 말고 세어야 한다고 말하며, 목록 옆에 적힌 총 개수는 참으로 유지해야 할 두 번째 대상이다 — 그리고 그것이 썩는 쪽이며, 다음에 보드가 하나 추가될 때 또 썩을 것이다. 목록을 세라.

같은 상수를 지니면서도 의도적으로 이 목록에 없는 것이 둘 있다. 동결된 참조 트리가 그것을 지니는 이유는 그것이 제품이 아니라 비교용 스냅샷이기 때문이다. 브리지 부트로더들은 부록 C 가 말하는 이유로 자기들만의, 동결되고 더 낮은 번호를 지닌다.

아래의 모든 항목은 그 목적으로 관리한 변경 이력이 아니라 그 사본들의 이력으로부터 도출되었으며, 각 항목은 선 위에서 무엇이 바뀌었는지를 말한다.

- **2.0.0** (2026-08-21) — 번호 그 자체. 두 계층이 함께 명령 집합 버전을 선언하기 시작했다. 디바이스 쪽에 `CMD_GET_VERSION(0x05)`이, 브리지 쪽에 `CMD_B_GET_VERSION(0xC7)`이 추가되었고, 브리지의 설정 명령 두 개가 SWD 게이트 아래로 — 어느 쪽도 SWD 핀을 구동하지 않으므로 마땅히 있어야 할 자리로 — 옮겨졌다.
- **2.1.0** (2026-08-26) — `CMD_FLASH_READ(0xF3)`가 예약에서 구현으로 바뀌었다.
- **2.2.0** (2026-08-26) — 부트로더 전용인 브리지 자체 갱신 블록 `CMD_UPD_*(0xE8-0xED)`와 그 `ERR_UPD_*` 오류 코드, 실행 중인 이미지의 역할을 위한 버전 선택자, 그리고 `CMD_MODE_BRIDGE_RESET(0xCB)`이 예약에서 구현으로. 대상: `0x00` 앱, `0x01` 부트로더, `0x02` CMSIS-DAP 페르소나(3.0.0, C.1 참고; 페르소나가 없는 브리지는 `ERR_BAD_ARGS` 로 답한다).
- **2.3.0** (2026-08-27) — `CMD_BRIDGE_CONF(0xCE)`. 설정마다 오피코드를 두는 대신 런타임 설정을 파라미터/값 쌍으로 실는 하나의 오피코드.
- **2.4.0** (2026-08-30) — 브리지 버전 명령의 하드웨어 식별 선택자. 어느 이미지에서든 보드 식별 정보와 리비전에 답한다. 그전까지 두 브리지 보드를 구별할 유일한 수단은 자유 문자열이었다.
- **2.5.0** (2026-08-30) — 코어 정지와 코어 재개가 플래시 오류 코드를 빌려 쓰는 대신 자기 오류 코드를 갖게 되었다. 빌려 쓰는 바람에 실측 시간이 낭비된 적이 있다(§ 4.4).
- **2.6.0** (2026-09-01) — 브리지의 옵션 바이트 명령에 옵션 바이트 마스크 비트 하나가 명명되었다.
- **2.7.0** (2026-09-03) — 리셋 선이 배선되지 않은 대상을 다루기 위한 브리지 설정 키.
- **2.8.0** (2026-09-04) — SWD 핀 스왑을 선택하는 브리지 설정 키.
- **2.9.0** (2026-09-04) — 핀 맵 설정 키. 값이 플래그가 아니라 의미를 실는 첫 키이며, 설정 값을 0 이나 1 로 뭉개던 것을 멈춘 호스트 측 수정과 함께 들어왔다. 그 수정이 없었다면 그 키의 두 번째 값이 첫 번째 값으로 전송되었을 것이다. 그러면 브리지는 첫 번째 맵을 고른 뒤 자신이 실제로 받은 그 값을 되돌려주므로, 응답은 도착한 프레임과 앞뒤가 맞고 호스트가 다른 맵을 요청했다는 사실은 선로 위 어디에도 드러나지 않았을 것이다.
- **2.10.0** (2026-09-08) — `CMD_FLASH_READ_EX(0xF5)`. 평범한 읽기가 보고할 수 없는 결함 횟수까지 함께 돌려주는 읽기.

- **2.11.0** (2026-09-09) — `CMD_TGT_POWER` 에 대한 쓰기가 브리지 설정 키 뒤로 옮겨졌다. 대상의 전원을 공급할 수 있는 보드가 기본적으로는 공급하지 않게 하기 위함이다.
- **3.0.0** (2026-09-11) — `CMD_MODE_BRIDGE_RESET` 이 세 번째 대상 `BRIDGE_RESET_TO_DAP(0x02)`을 얻었다: 브리지 보드가 평소의 CDC 페르소나 대신 CMSIS-DAP v1 HID 디버그 장치로 재부팅한다. C.1 참고. 그리고 `CMD_B_GET_CAPS (0xC2)`가 예약에서 구현으로 바뀌었다: 응답하는 펌웨어가 무엇을 할 수 있는지를 말하는 기능 워드(`BCAP_*`)로, 호스트가 더 이상 보드별 표를 들고 다니지 않는다. C.2 참고.
- **3.1.0** (2026-09-13) — 현재, `CMD_SET_TARGET` 이 다섯 번째 모델 `CWM30C8 (0x04)`을 받아들이고, 그 응답이 대상의 프로그램 페이지와 소거 섹터를 싣는다. 그 패밀리는 이전의 모든 패밀리가 128 B / 4 KB 였던 자리에서 256 바이트 페이지를 프로그램하고 8 KB 섹터를 소거하기 때문이다. 플래시 블록의 세 오피코드 (`0xF0-0xF2`)는 번호를 유지한 채 하는 일을 말하는 이름으로 바뀌었다 — 단위는 오피코드의 것이 아니라 선택된 대상의 것이다. C.3 참고. 부록 D 에 네 번째 시험 펌웨어가 더해졌다.

기록이 끊기는 곳. 기록은 2.0.0 아래에서 끊기며, 복원할 것도 없다. 명령 집합 버전은 2026-08-21 에 도입될 때 이미 2.0.0 으로 번호가 매겨졌고, 1.x 를 선언한 구현은 존재한 적이 없다. 참조 펌웨어 자신의 프로토콜 문서에 나오는 v1.x 번호는 그 문서 자신의 개정 번호이며 명령 집합 버전이 아니다. 그것을 이 목록의 더 이른 항목으로 읽는 것은 잘못이다. 다만 그 번호들이 비어 있는 것은 아니다. 그 문서는 선 위에 보이는 변경이 들어올 때 — 명령이 추가되거나 제거되거나 번호가 바뀌었을 때, 프레임 배치가 바뀌었을 때, 기본 동작이 바뀌었을 때 — 자기 번호를 올리고 내부 구현 변경에는 올리지 않으며, 그 v1.0 부터 v1.5 까지의 항목은 실제 변경들을 기록하고 있다. 로그 오피코드가 옮겨졌고, 버스트 페이로드의 순서가 바뀌었고, 동작 모드 하나가 폐지되었고, 기본 모드가 알 수 없는 명령에 답하던 방식이 오류에서 침묵으로 바뀌었고, 튜닝 명령의 페이로드 앞에 필드가 하나 늘었다. 이들은 명령 집합 번호 부여보다 완전히 앞서는, 한 제품에서의 선 위 변경을 서술한다.

이 목록의 두 가지 성질은 분명히 적어 둘 값어치가 있다. 2.0.0 이후의 모든 증가가 MINOR 였고, 그 하나하나가 브리지 대역에서 일어났다. 디바이스 대역은 2.0.0 이후 바뀌지 않았으며, 그래서 2.0.0 에 맞춰 만든 디바이스와 2.11.0 에 맞춰 만든 호스트가 그 디바이스가 가진 모든 오피코드에 대해 여전히 일치한다. 이는 번호가 매겨진 구간에 대한 진술일 뿐 그 이전에 대해서는 아무것도 말하지 않는다. 위의 v1.x 항목들이 기록하듯 디바이스 대역은 번호 부여가 시작되기 전 몇 주 동안 거듭 번호가 다시 매겨졌고, 그것이 이 번호 부여가 존재하는 이유의 일부이다. 그리고 모든 증가가 선 위의 변경인 것도 아니다 — 2.9.0 은 일부가 호스트 측 장부이다 — § 9.3 의 규칙은 위 목록의 모든 사본이 함께 움직인다는 것이지 증가마다 오피코드가 하나씩 늘어난다는 것이 아니기 때문이다.